Evaluating Reinforcement Learning Algorithms for Portfolio Trading

A Reinforcement Learning Study on the S&P 500 ETF (SPY)

Fiyinfoluwa Akano
6962514

MSc Artificial Intelligence
School of Computer Science and Electronic Engineering
Faculty of Engineering and Physical Sciences
University of Surrey

Chapters 3–5 — September 1, 2026

This is a preview build of Chapters 3–5 for supervisor review.
Citations, front matter and remaining chapters follow separately.

Contents

Chapter 3

Methodology & Mathematical Formulation

This chapter explains how the portfolio problem can be treated as a Markov Decision Process (MDP) [1, 2], i.e the trading process is treated as a sequence of step-by-step decision-making problems. The objective is to develop an environment in which an autonomous trading agent can learn an optimal decision-making behaviour by interacting with a simulated market. At each step, the portfolio system observes current market and portfolio conditions, takes an action such as buying, selling, or holding, receives a result based on how the portfolio's wealth changed, and then moves to the next step.

The chapter then explains the two main learning methods used in the project: REINFORCE [3]—the Monte-Carlo policy gradient, which learns by improving its strategy based on the rewards it receives —and Deep Q-learning [4, 5], which learns which actions are likely to give the best results. Finally, PPO is introduced as an additional method in the ablation row to compare against the other approaches, as it is designed to make the policy learning process more stable.

Table 3.1 collects every symbol used for reference.

Table 3.1: Notation used throughout Chapters 3–5.

Symbol	Meaning
$s_t \in \mathbb{R}^8$	state observed by the agent at trading day t, containing the eight features used by the trading system (Table 3.3)
$a_t \in \{0, 1, 2\}$	action selected by the agent: Hold, Buy, or Sell.
P_t	asset price at day trading t;
r_t	reward received after an action, calculated from the change in portfolio value after transaction costs.
C_t	amount of cash available to the agent at trading day t
h_t	number of (fractional) units of the asset held at trading day t.
$W_t = C_t + h_t P_t$	Total portfolio value, calculated as untouched cash plus the current value of the asset held;
C_0	initial cash available at the beginning of an episode.
c	proportional transaction fee for taking a trade, set to 0.05% (c = 0.05)
$\Delta = 0.1\ C_0$	fixed monetary size of each trade
γ	discount factor used to determine the importance of future rewards
τ	a trajectory (one complete trading episode), corresponding to one month of daily trading; related to t & T
T	episode length in days (with $T \approx$18–23)
t	1 trading day

Symbol	Meaning
$G(\tau)$	The total return obtained over a complete episode
G_t	The accumulated future reward from time t onwards.
$\pi_\theta(a \mid s)$	policy network that determines the likelihood of choosing action a given the current state s.
$Q_\varphi(\mathrm{s}, a)$	The Q-network, which estimates the value of taking a particular action in a given state; frozen target copy
Q_{φ^-}	A fixed copy of the Q-network used as a target during Deep Q-learning.
$J(\theta)$	objective function representing the expected return of the policy
ε	exploration rate used by the (ε-greedy) action-selection strategy

3.1 Overview: Trading as a Markov Decision Process

Reinforcement Learning (RL) trains an agent to make sequential decisions by interacting with an environment. The environment is modelled as an MDP where decisions are made at regular time steps, represented by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$: at step t the agent observes the current state $s_t \in S$, selects an action $a_t \in A$, receives a numerical reward based on the outcome r_t, and the environment moves to the next state s_{t+1} according to the current state and the action selected by the agent $P(s_{t+1}|s_t, a_t)$.

In this project, the trading environment is modelled in the same way, providing a formal description of how the trading agent interacts with the market over time; one time step t corresponds to one trading day.

The state s_t represents all information available to the agent on day t to fully capture its position within the trading episode. This includes the selected market features, the agent's current portfolio position, and its available cash.

Based on this state, the agent selects one of three possible actions $a_t \in \{0,\ 1,\ 2\}$: *Hold the current position, Buy a fixed amount, or Sell a fixed amount.* The selected trade is then executed, with transaction fees c applied where appropriate.

After taking an action, the environment calculates the change in the agent's total portfolio value and returns it as the reward r_t.

The environment advances to the next trading day s_{t+1}, and the process repeats throughout the episode τ, allowing the agent to update its policy π_θ or value estimates Q_ϕ based on the experience collected.

The interaction can therefore be represented as: $s_t \rightarrow a_t \rightarrow r_t, s_{t+1}$

Figure 3.1 illustrates the interaction between the trading agent and the environment.

To make the MDP formulation easier to understand, Table 3.2 compares the main components of the trading task with those of a familiar game-playing task, Flappy Bird, which was used initially to understand Reinforcement Learning.

Table 3.2: Mapping a popular game MDP to the portfolio trading MDP.

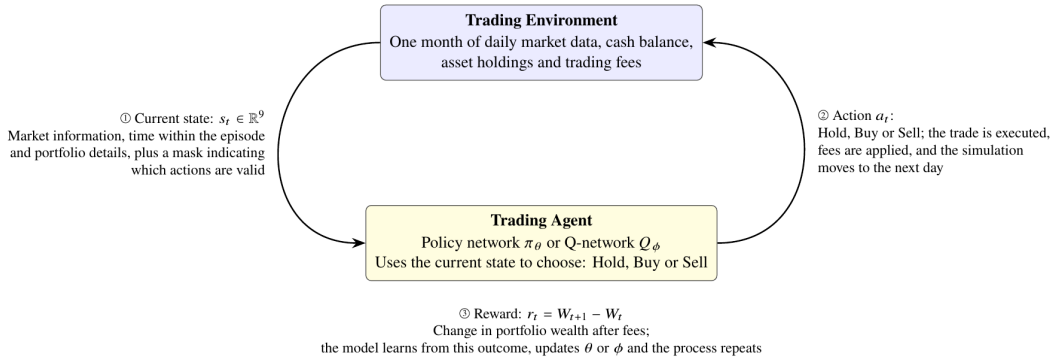

Figure 3.1: The agent–environment interaction loop for the trading MDP. One cycle corresponds to one trading day.

MDP component	Flappy Bird	Trading (this work)
State s_t	Bird's position and speed, together with the position of the next pipe	Market information and the agent's current portfolio position
Action a_t	Flap or do not flap	Hold, Buy, or Sell
Reward r_t	Survival score based on how long the bird remains in play	Change in portfolio wealth after trading fees
Episode τ	One complete game session	One calendar month of daily trading
Terminal rule	The game ends when the bird crashes or hits the ground	The episode ends at the final trading day, with any remaining position sold

This analogy comparison also highlights an important design principle used throughout the project. In Flappy Bird, the state sees the whole world by containing enough information relevant to the next decision, such as the bird's position and movement and the location of the pipes. The trading state follows the same principle by including the relevant information available to the agent at each trading step. This ensures the agent has access to the information it is allowed to observe when making decisions, without being given future information. Section 3.3 discusses the design of the trading state in more detail.

3.2 Episode Definition and Data Structure

3.2.1 Episode definition (τ)

Each episode (τ) represents one calendar month of daily closing prices, from P_0 *to* P_T (P_0, $P_1, \ldots, P_T$) where T is typically between 18 and 23 trading days per month, excluding public holidays when markets aren't open and weekends of market inactivity ($T \approx 18 - 23$). At the beginning of each episode, the agent starts with cash and no asset holdings. It can then trade throughout the month based on the available information. At the end of the final trading day t, any remaining position is automatically sold. This ensures that no position is carried over from one episode to the next.

3.2.2 Independence of Trading Episodes

For the purposes of the experiment, each monthly episode is treated as a separate and independent trading period. This is similar to treating each game session in Flappy Bird as a separate episode.

However, this is a simplification because market conditions in consecutive months are not truly independent. For instance, events and trends in one month do influence the market in the following month. The resulting data are therefore not independent over time.

Modelling these temporal relationships in greater detail would require time-series or partially observable approaches, which are outside the scope of this project. The implications of this assumption are discussed further in the conclusion.

3.2.3 Choice of Episode Length

Monthly episodes were selected to provide a balance between the length of each trading period and the total number of training examples available.

A single year of daily data provides approximately 12 monthly episodes and around 250 daily trading decisions. Using eight years of data from 2018 to 2025 gives 96 monthly episodes and approximately 2,000 daily transitions. These episodes are divided chronologically into 58 months for training and 38 months for testing. The data are not shuffled, which prevents information from later periods from being used during training and helps avoid look-ahead leakage.

The same environment could also be adapted to use minute-level data in future work. A typical trading session T would contain approximately ~ 390 one-minute intervals in this case, providing a much larger number of possible decision points. This would not require a fundamental change to the environment because the framework is designed to work across different episode lengths. However, minute-level trading is outside the scope of the current study and is therefore left as a potential area for future development.

3.2.4 Selection of the Historical Period

The experiments use daily market data from 2018 to 2025 as a fixed period. Earlier data, including data from 2016 and before, were not included in the main experimental period because the study aims to evaluate the learning algorithms under relatively recent and varied market conditions. These selected periods contain a range of market conditions, including relatively stable periods in 2018, the market decline associated with the COVID-19 pandemic and its subsequent recovery, the later period of increased inflation and interest rates, and the war in Ukraine in 2022, which caused weaker market conditions. As a result, the dataset is not limited to a single type of market condition, providing a useful range for evaluating the behaviour of the portfolios trading agents without extending the dataset into substantially older market years.

The exclusion of earlier data does not imply that those years are unsuitable for financial modelling. Instead, the decision was to keep the experimental dataset focused and consistent with the study's scope. Including older data, such as 2016 or earlier, would increase the number of observations but could also introduce market conditions that differ considerably from those in the selected period; for instance, older microstructure and fee regimes that differ slightly.

Earlier historical data could therefore be considered in future work as an additional robustness test to examine how well the trained agents generalise across different historical market regimes.

3.2.5 MDP Limitations

Although the portfolio trading problem is modelled as an MDP, real financial markets are much more complex and cannot be assumed to be fully observable.

The Markov assumption means that the current state should contain all the information the agent needs to make its next decision. In a real market, however, many external factors can affect price movements that are not directly available to the agent. These include:

- economic announcements;
- investor sentiment;
- order-book activity;
- geopolitical events;
- movements in related assets.

As a result, the MDP used in this dissertation should be viewed as a simplified representation of the portfolio trading problem rather than a complete model of how real financial markets operate.

The purpose of the state design is therefore not to capture every factor that may influence an asset's price, but to provide the agent with the key information needed to make decisions within the controlled trading environment used for the experiments.

3.3 State Representation Design

In reinforcement learning, how information is presented to the agent plays an important role in the quality of decisions it can learn. The state representation must provide enough information for the agent to understand the current situation and choose an appropriate action.

In this portfolio trading problem, this requires capturing information about both the market and the agent's own portfolio. The agent needs to know how the asset has been moving, but it also needs to know how much cash is available, whether it currently holds the asset, whether that position is profitable, and how much time remains before the trading period ends.

A state that leaves out important information can make it difficult for the agent to learn effective strategies, even if the learning algorithm is suitable for the task. This is because the agent can make decisions only using the information provided. Therefore, the state representation in this study was developed gradually by first examining simpler representations and then adding information as needed to address specific limitations of the previous representation.

The objective was not to create an unnecessarily complex state representation with many variables, but to design a compact representation containing all relevant information needed for decision-making within the defined portfolio trading environment, while avoiding features that simply duplicate information already available from other variables

3.3.1 Information Available to the Agent

As discussed in Section 3.1, the state should see the whole by providing the agent with information relevant to its decision, as the Flappy Bird state does. In a trading environment, this information can be divided into two halves: the agent's own portfolio and the wider market. These two areas behave very differently and should therefore be considered separately when designing the state.

The first area is the agent's own portfolio. This includes information such as the amount of cash available, how much of the asset it owns, what it paid to enter a trade position, and the time remaining in the trading period. The agent controls this half; it is not hidden or estimated, and it can be completely represented. Given the sequence of the asset prices and the agent's actions, the portfolio can be updated directly using the trading environment's rules.

The second area is the market. Here, complete information about the market is impossible in principle and cannot be provided to the agent because many factors affecting an asset's price are not directly observable from the available price data. For example, the agent cannot observe other market participants' decisions, expectations, or intentions. Similarly, information such as news, market sentiment, and macroeconomic events may influence prices without appearing directly in the closing price at the end of the episode.

Adding more features calculated from past prices can give the agent additional information about recent market behaviour, but it cannot reveal everything happening in the market, as the future remains only partly predictable from them. Future price movements therefore remain uncertain, even when a relatively large number of historical features are included.

The Flappy Bird analogy therefore needs some qualification, as its world is finite and deterministic, so a handful of numbers genuinely is sufficient. A market's world is neither. The attainable goal is a state representation that is complete on the agent's portfolio side and rich enough on the market side that adding further features no longer changes what the agent does.

For this reason, the state representation used in this study makes an important note. The portfolio information can be represented completely because the environment directly controls and records it, whereas market information can only approximate the wider market.

Whether the additional market features provide useful information is ultimately an empirical question, which is considered when evaluating the state representation later in the dissertation.

3.3.2 Criteria for a Suitable Trading State

It is tempting to design a state by asking which features look useful, but this is a weak approach as it says nothing about whether the object being trained is an MDP at all.

Before deciding on the final state representation, three conditions were considered. These provide a basis for deciding whether the proposed state contains the information required for the portfolio trading problem.

1. Reward measurability

The reward should be calculable from information available to the agent and the resulting transition. If the reward depends on information not contained in the state or the transition, the agent is effectively being evaluated using information it cannot observe.

The primary reward used in this study is based on the change in portfolio wealth:

$$r_t = W_{t+1} - W_t$$

where:

$$W_t = C_t + h_t P_t$$

Here, C_t represents the available cash,

h_t represents the number of asset units held, and

P_t represents the current asset price.

To make these values comparable across different starting capital levels, the final state uses normalised cash and asset exposure:

$$c_t = \frac{C_t}{C_0}$$

and:

$$v_t = \frac{h_t P_t}{C_0}$$

where C_0 represents the initial capital.

These two features can be combined as:

$$c_t + v_t = \frac{C_t}{C_0} + \frac{h_t P_t}{C_0} = \frac{W_t}{C_0}$$

This means the agent's current portfolio wealth, relative to its initial capital, can be obtained directly from the state, ensuring the reward can be calculated from the observed transition without relying on information or variables hidden from the agent.

Consequently, the wealth-change reward can be written as:

$$W_t = C_0[c_t + v_t]$$

$$r_t = C_0 \left[(c_{t+1} + v_{t+1}) - (c_t + v_t)\right]$$

The reward can therefore be calculated from the current state, action, and next state $(s_t,\ a_t, s_{t+1})$ with no additional hidden portfolio variable.

2. Transition consistency

The state should also contain enough information for the next state to be determined from the current state, the selected action, the next observed market price, and the trading environment rules.

This matters because the agent should not need to access hidden historical variables to describe how its portfolio changes after an action.

For example, the current portfolio exposure can be used to determine the position's value, while the current unrealised profit/loss contains information about the position's entry price. Together with the current price and the environment's trading rules, these quantities provide the information required to update the portfolio after an action.

This keeps the state representation consistent with the defined trading environment rather than requiring the environment to retrieve additional portfolio history that is not represented in the state.

3. Decision sufficiency

The state should contain enough information for the agent to make a decision based on the current trading situation without ambiguity. If two different situations look identical to the agent but require different actions, then the state is missing important information, making the representation incomplete.

For example, consider the following two situations:

- the asset price has increased by 2% today, and the agent does not hold the asset;
- the asset price has increased by 2% today, but the agent holds a position that is currently making a loss.

In both cases, the observed price movement is the same. However, the most appropriate action may be different. In the first situation, the agent may consider opening a position, whereas in the second, it may consider reducing or closing its existing position to minimize its losses.

The state representation must therefore include information about both the market and the agent's own portfolio.

3.3.3 Development of the State Representation

The initial state representation included only basic information about market movement and the agent's portfolio. While this provides a simple starting point, it does not give the agent enough context to fully understand its trading situation.

For instance, recent price changes alone do not tell the agent whether it owns the asset, how much capital remains available, or whether an existing position is in a profit or loss. Without this information, the agent may interpret the same market condition in the same way, even when the appropriate action depends on its current portfolio situation.

To address this limitation, the state representation was gradually expanded in multiple stages. Each stage introduced additional information to address a specific weakness identified in the previous representation.

Stage 1: Price-only representation

The first and simplest representation provided the agent with only the recent return of the asset:

$$s_t^{(0)} = [\Delta P_t]$$

where:

$$\Delta P_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

This feature represents the immediate change in the asset price between two consecutive time steps.

However, this representation is insufficient because the same market movement can lead to different trading situations. For example, a positive daily return in price could occur in two different situations:

- the agent does not currently hold the asset and may consider entering a position;
- the agent already holds a large position and may consider reducing its exposure.

Although the market information is identical in both cases, the most suitable action may be different. The first limitation of the state is therefore that the agent cannot see its own portfolio.

This highlights the need for the state representation to include information about the agent's own portfolio state in addition to market movement.

Stage 2: Portfolio-aware representation

The second representation extends the state by including basic information about the agent's portfolio:

$$s_t^{(1)} = [\Delta P_t,\ C_t,\ h_t]$$

where:

- (C_t) represents the amount of available cash in the portfolio;
- (h_t) represents the number of asset units currently held in the portfolio.

By including portfolio information, the agent can distinguish between different portfolio situations and make decisions based on its available resources. For example, it can identify whether it has enough capital to open a new position or whether it already holds the asset.

However, this representation is still incomplete. The number of units held alone does not fully describe the financial size of the current position. For instance, holding ten units of an asset has a very different financial impact when the asset price is £10 compared with when it is £1,000.

Furthermore, the agent cannot also determine whether its current position is profitable because it does not know the purchase price. It also lacks information on how much time remains in the trading episode ends, which may influence decision-making.

Therefore, further information is required to create a more complete representation of the agent's trading environment.

Stage 3: Position-aware representation

To address the limitations of the previous representation, the state was extended to include information about the size of the current position opened and its performance. This allows the agent to better understand what it owns and the financial impact of its current position.

The following variables were added:

$$s_t^{(2)} = [\Delta P_t, c_t, \upsilon_t, PnL_t]$$

1. Normalised cash (c_t)

$$c_t = \frac{C_t}{C_0}$$

where C_t represents the available cash in the portfolio at time t, and

C_0 represents the portfolio's initial capital.

This feature indicates the proportion of the original capital that remains available for future decisions. By using a normalised value, the information remains consistent regardless of the starting account size.

2. Normalised exposure (v_t)

$$v_t = \frac{h_t P_t}{C_0}$$

This represents the current market value of the open position relative to the initial capital.

This gives the agent a clearer understanding of its actual asset exposure, unlike simply recording the number of units held. For example, the same number of units can represent a small or large investment depending on the current asset price.

The normalised cash and asset exposure can also be combined to estimate the agent's current wealth:

$$c_t + v_t = \frac{W_t}{C_0}$$

where W_t represents the current portfolio wealth.

This allows the agent to reconstruct its current wealth from the cash available, and the value of the asset currently held in the portfolio.

3. Unrealised profit/loss (PnL_t)

$$PnL_t = \frac{P_t - \overline{P}_t^{entry}}{\overline{P}_t^{entry}}$$

Where $\overline{P}_t^{entry}$ represents the average price at which the current position was opened.

This feature allows the agent to identify whether its current position is profitable or losing. Without this information, the agent cannot distinguish between different situations that look identical from a market perspective.

For example, holding an asset after a price increase may be profitable if the asset was purchased at a lower price. However, holding the same asset could instead be losing money if the position was entered at a higher price before the price declined.

Including unrealised profit/loss enables the agent to recognise this difference and make more informed decisions.

Stage 4: Time-aware representation

Trading decisions are also affected by the amount of time remaining within the trading episode

In this environment, each episode represents a fixed trading period, meaning that all open positions must be closed before the final timestep. Therefore, an action may have different consequences depending on when it is taken.

For example, buying an asset at the start of an episode gives the position more time to potentially increase in value compared with buying the same asset shortly before the episode ends, when the position must soon be closed.

To capture this information, the state representation includes the remaining time left in the episode:

$$s_t^{(3)} = \left[\Delta P_{t_t}, c_t, v_t, PnL_t, \tau_t\right]$$

$$\tau_t = \frac{t}{T}$$

Where T represents the total length of the trading episode.

A value close to 0 indicates that the episode has only recently started, while a value close to 1 indicates that the end of the episode is approaching.

This feature lets the agent to take the remaining trading episode into account when selecting an action and adjust its decisions accordingly. For example, the agent may be more willing to take long-term positions early in the episode, while becoming more cautious as the final timestep approaches.

3.3.4 Final State Representation

Although the previous stages successfully introduced information about the agent's portfolio and position, they provide limited understanding of the broader market conditions. A trading decision should not depend only on the most recent price change, as different market situations exist, producing similar short-term movements.

For this reason, the agent also needs information that helps identify the current market conditions to be able to distinguish between them, such as:

- whether the asset is experiencing a short-term upward or downward trend;
- whether the market is currently stable or highly volatile;
- whether the current price is above or below its recent average level.

Therefore, additional market-related features are included to give the agent a more complete view of the trading environment.

The final state representation is defined as:

$$\boxed{s_t^{(4)} = \left[\Delta P_t, mom_t, vol_t, gap_t, rng_t, \tau_t, c_t, v_t, PnL_t\right] \in \mathbb{R}^9}$$

The following variables were added:

1. Momentum

$$mom_t = \frac{P_t - P_{t-k}}{P_{t-k}}$$

Where $k = 5$ previous days ~ 1 week timestep

Momentum measures the direction of price movement over the previous five timesteps. It gives the agent a longer view of recent price direction and helps the agent differentiate between a temporary price movement from a more consistent trend developing over several timesteps.

2. Volatility

$$vol_t = \sigma(\Delta P_{t-k+1}, ..., \Delta P_t)$$

Where σ represents the standard deviation of recent returns and $k = 5$.

Volatility measures how much recent returns have varied using Standard Deviation.

This feature helps the agent recognise differences in market uncertainty. For example, a 2% price increase during a relatively stable period is different from a 2% increase during a period of large and frequent price movements, indicating a riskier situation.

3. Price trend position

$$gap_t = \frac{P_t}{MA_{m,t}(P)_t} - 1$$

where $MA_{m,t}(P)_t$ represents the simple moving average of the asset's closing price over the previous ($m = 20$) days ~ 1-month timesteps.

The simple moving average $MA_{m,t}(P)_t$ can be calculated as:

$$MA_{m,t}(P)_t = \frac{1}{m} \sum_{i=0}^{m-1} P_{t-i}$$

This feature shows whether the current price is above or below its recent average. A positive value indicates that the current price is above its recent average level, while a negative value indicates that it is below it, helping the agent identify potential trends or deviations from normal price behaviour.

4. Normalised Trading Range

The normalised trading range is based on the Average True Range (ATR):

$$rng_t = \frac{ATR_{n,t}}{P_t}$$

The True Range at timestep t is:

$$TR_t = \max(H_t - L_t,\ |H_t - C_{t-1}|,\ |L_t - C_{t-1}|)$$

where:

- H_t = current high price
- L_t= current low price
- C_{t-1}= previous closing price

The ATR is then calculated from the recent True Range values. Using the simple rolling arithmetic mean procedure, the calculation after the initial ($n = 14$) days ~ 2-week timesteps value can be expressed as:

$$ATR_{n,t} = \frac{1}{n} \sum_{i=0}^{n-1} TR_{t-i}$$

$$ATR_{14,t} = \frac{1}{14}\sum_{i=0}^{13} TR_{t-i}$$

The *Average True Range (ATR)* measures the size of recent price movement using the asset's highest price point, lowest price point, and closing prices. Dividing the ATR by the current price creates a normalised measure, making it easier to compare price movements across different price levels.

The features are organised into four categories:

Table 3.3a: Feature Categorization

Category	Features	Purpose
Market behaviour	$\Delta P_t, mom_t, vol_t, gap_t$	Describe recent market conditions and price behaviour
Portfolio information	c_t, v_t	Describe available capital and current exposure
Position information	PnL_t	Describe performance of any position opened
Time information	τ_t	Represent the remaining trading episode

Table 3.3b: Full State Definition

Table 3.3b: The nine-feature state representation ($s_t \in \mathbb{R}^9$) used in the main experiments. C_0 denotes the initial cash, h_t the number of asset units held, and $W_t = C_t + h_t P_t$ the portfolio wealth at time t. the mark-to-market wealth. The feature timestep lengths are set to (k=5) for momentum and volatility, (m=20) for the moving-average reference, and (n=14) for the Average True Range.

#	Feature	Definition	What it tells the agent
Market			
1	ΔP_t	P_t - P_{t-1}/ P_{t-1}	The most recent price movement
2	mom_t	P_t - P_{t-k}/ P_{t-k}	whether that recent price movement is part of a short-term trend
3	vol_t	$\sigma(\Delta P_{t-k+1}, ..., \Delta P_t)$	how much the asset price has been varying recently
4	gap_t	P_t / $\mathrm{MA}_m(\mathrm{P}) - 1$	how far the current price is from its recent average
5	rng_t	$\mathrm{ATR}_n(\mathrm{P})$ / P_t	The size of the recent price ranges, which closing prices alone cannot show
Portfolio			
6	c_t	C_t / C_0	how much of the initial cash remains available as cash
7	v_t	$h_t P_t$ / C_0	The current value of the open position relative to the initial capital
Position			
8	PnL_t	$(P_t - \bar{P}_{tentry})$ / $\bar{P}_{tentry}$	whether the open positions are in profit or loss

#	Feature	Definition	What it tells the agent
Time			
9	τ_t	t / $T \in [0, 1]$	how much time remains before the trading episode ends

3.3.5 Feature Redundancy and Excluded Information

The state was not designed by adding every potentially useful market indicator. Features were included only where they provide information that cannot already be recovered from the other variables in the state.

This is important because adding a feature that simply repeats existing information increases the size of the input without providing the agent with anything new.

The following features were excluded:

1. Running portfolio return

For example, the portfolio's current return relative to its starting capital
does not need to be included as a separate feature because it can already be calculated from normalised cash and asset exposure:

$$c_t, v_{t-1}$$

Therefore, adding running portfolio return would duplicate information already available in the state.

2. Raw position size

The raw number of units held, h_t, is also not included in the final state. The financial value of the position is more directly represented by:

$$v_t = \frac{h_t P_t}{C_0}$$

This provides information about the actual exposure rather than only the number of units.

3. Raw asset price

The raw asset price is also not required as a separate feature. The state instead uses returns and normalised values, which describe price movements without depending on the standard price level.

The general principle is therefore to include features that provide useful information while avoiding variables that can already be reconstructed from information available elsewhere in the state.

3.3.6 The Risk-Aware Extension: Drawdown State

The main state representation contains the information needed for the change in wealth reward. However, it does not describe the history of the portfolio's losses. This matters for risk-sensitive experiments because the current portfolio value alone does not show how much the portfolio may have fallen from a previous high.

This becomes important when considering losses and the portfolio's previous performance because current portfolio wealth alone does not show how the portfolio reached its current value. For example, two portfolios can have exactly the same wealth at the current timestep but may have followed very different paths:

- one portfolio may have increased steadily over time;
- another may have suffered a large loss before recovering to the same level.

To address this, an additional feature representing portfolio drawdown is introduced as a separate risk-aware extension:

$$dd_t = \frac{\max_{u \leq t} W_u - W_t}{\max_{u \leq t} W_u}$$

where dd_t represents the current decline in portfolio wealth from the highest wealth level reached so far during the episode.

A value of zero means that the portfolio is currently at its highest wealth level reached so far in the episode. A larger value indicates that the portfolio has fallen further below its previous peak.

For this reason, drawdown is treated separately from the main state representation. This makes it possible to evaluate whether explicitly providing information about portfolio loss history affects the agent's behaviour and performance.

The resulting risk-aware state is therefore:

$$s_t^{risk} \in R^{10}$$

where:

$$s_t^{risk} = [s_t, dd_t]$$

Since the original state s_t contains nine features, adding drawdown produces a ten-dimensional risk-aware state.

3.3.7 Feature Calculation and Information Timing

The timing of feature calculation matters because the agent should receive only the information available when making its trading decision. Using future information would give the agent an unfair advantage and could lead to overly optimistic results.

For this reason, the market features are calculated using information available at or before timestep (t). The current price and past prices can be used to create the state, but the agent must not use future prices when it decides its action (a_t).

The same principle applies to the features that use historical windows. Momentum uses the previous ($k = 5$) timesteps, volatility is calculated from recent returns, the moving average uses the ($m = 20$) previous prices, and the normalised trading range is based on the ($n = 14$) day ATR. In this study, these parameters are set according to the values reported in Table 3.3b.

These look-back periods are used to provide different views of recent market behaviour. Momentum provides information about the short-term direction of recent price, volatility shows how much

recent returns have been changing, the moving-average gap shows whether the current price is above or below its recent average, and ATR provides information about the size of recent price ranges.

Therefore, each state is based only on information that is available at the time the decision is made. This prevents future market information from entering the agent's observation and helps avoid look-ahead bias, where a model appears to perform well because it has indirectly been given information that would not have been available during actual trading.

3.3.8 State Representation Limitations

Although the final state representation gives the agent a wider range of information, the trading environment is still a simplified representation of a real financial market.

The agent does not have access to several types of information that can influence real-world trading decisions, including:

- order flows;
- market sentiment;
- financial news;
- macroeconomic conditions;
- relationships between the asset and other financial assets.
- The actions and intentions of large financial institutions.

These limitations mean that the state should not be interpreted as a complete description of the real financial market. Because this relevant information cannot be represented in the environment, it is better to describe this as a *partially observable Markov decision process (POMDP)* rather than a fully observable one.

However, this dissertation does not aim to replicate every aspect of real-world financial markets. The purpose is to compare and evaluate reinforcement learning algorithms within a controlled trading environment for an asset.

The proposed state representation therefore provides sufficient information for the defined task and can be considered complete with respect to the information used by the defined trading environment, which includes recent market behaviour, the current portfolio position, the performance of its current position, and the amount of time remaining in the trading period, while acknowledging that the model does not capture all factors present in real-world trading.

3.3.9 Summary

This section developed the state representation by gradually adding information to address limitations where simpler representations were insufficient. The process began with the smallest state that could represent a trading decision and was then expanded to include information the agent needed to distinguish different market and portfolio situations. The final state was not selected because it contained a particular number of variables, but because its components collectively describe the market conditions, portfolio exposure, position performance, and remaining trading episode available within the defined environment.

Table 3.4: The state-design ladder. Each rung resolves the limitation named in the row above it. ΔP_t is the one-step return, c_t cash and v_t exposure as fractions of initial capital, h_t the units held, τ_t the episode

clock and dd_t the fall from peak wealth.

Stage	State	Limitation addressed by the next stage
S_0	$[\Delta P_t]$	The agent can observe recent price movement but has no information about its own portfolio. It cannot tell whether it already holds the asset, or whether it has enough cash to open a position.
S_1	$[\Delta P_t, c_t, h_t]$	The agent now knows its available cash and the number of units it holds but not what they are worth. For example, ten units could represent a small or large investment depending on the asset price.
S_2	$[\Delta P_t, c_t, v_t, PnL_t]$	The agent can now determine the value of its position and tell whether its in profit or loss, but it does not know where it stands in the month. A position opened at the beginning of the month can therefore appear identical to one made immediately before the required closing of the position at the end of the month.
S_3	$[\Delta P_t, c_t, v_t, PnL_t, \tau_t]$	The agent's own circumstances are now fully represented, but its view of the market is still limited to the most recent price return. It cannot tell a stable trend from a reversal, or between relatively stable and highly volatile conditions.
S_4	adds momentum, volatility, distance from trend, and daily range (Table 3.3)	The state provides a broader representation of the market while retaining the portfolios information. It is treated as the main state used in the experiments.
S_{risk}	adds dd_t	Adds information about the portfolio's previous losses from its highest wealth level. Required when the experiment considers risk history

A separate tenth feature, drawdown, is used for the risk-aware experiments. It is kept outside the core state because it contains information about the portfolio's previous performance and therefore depends on the path taken by the portfolio, rather than only on its current condition.

The main state representation provides the basis for defining the available actions and applying the reinforcement learning algorithms described in the following sections.

3.4 Action Space Design

3.4.1 Overview

The action space defines the choices available to the trading agent at each timestep. Its design matters because it determines how the agent interacts with the market and what its learned policy means.

A poorly designed action space can introduce unnecessary complexity or create unrealistic trading behaviour. For example, allowing the agent to choose any number of shares to buy or sell adds

learning challenges: the agent must simultaneously learn when to trade, which direction to take, and how much to trade. This can make it difficult to determine whether poor performance results from an ineffective trading strategy or from difficulty learning appropriate position sizes.

To keep the focus on the trading decisions made by the reinforcement learning algorithms, this study uses a simplified discrete action space. The agent decides on the action (trading direction), while the environment determines the amount traded.

This separates the trading problem into two decisions:

1. When and in which direction to trade — this is the trading policy being learned by the reinforcement learning algorithm.
2. How much capital to trade — this is fixed by the environment and treated as an experimental setting.

This approach keeps the action space simple and makes it easier to compare the learning algorithms on the same portfolio trading task. It also means that differences in performance can be interpreted mainly in terms of the agents' ability to learn trading decisions, rather than their ability to learn position sizing.

3.4.2 Trading Actions

The action space is defined as:

$$A = \{0, 1, 2\}$$

where:

Action	Description
0	Hold the current position
1	Buy a fixed amount
2	Sell a fixed amount

At each timestep, the agent selects one of these three actions using the information available in the current state.

1. Hold Action

The hold action leaves the current portfolio unchanged:

$$a_t = 0$$

No transaction takes place, meaning:

- the available cash remains unchanged;
- the number of asset units held remains unchanged;
- no transaction fee is charged.

This allows the agent to remain in its current position when it does not consider buying or selling to be appropriate. For example, it prevents unnecessary trading when market conditions are favourable.

2. Buy Action

The buy action increases the agent's exposure to the asset by purchasing a fixed monetary amount:

$$a_t = 1$$

The value of a fixed trading amount is defined as:

$$\Delta = 0.1C_0$$

Where:

C_0 is the initial capital, and

Δ represents the amount allocated to one transaction.

The number of units purchased is therefore:

$$\Delta h_t = \frac{\Delta}{P_t}$$

where P_t is the asset price at timestep (t), allowing fractional ownership of the asset similar to a real-world portfolio.

Using a fixed monetary amount means that each buy action represents the same proportion of the initial capital. This keeps the buy sizing consistent throughout the dataset, even when the asset price changes.

3. Sell Action

The sell action reduces the current position by a fixed trading amount:

$$a_t = 2$$

The number of units sold is:

$$\Delta h_t = -\frac{\Delta}{P_t}$$

provided that the agent holds enough units of that asset for this transaction to be valid.

This lets the agent reduce its exposure gradually, rather than forcing it to close the entire position at once.

As a result, the agent can build or reduce its position over several decisions while still operating within a simple three-action framework.

3.4.3 Fixed Trading Amount

A key design choice in this environment is to use a fixed monetary amount for each transaction rather than a fixed number of shares.

In a conventional stock trading environment, an agent may be allowed to buy or sell a whole number of shares. However, this can make the size of the trade depend heavily on the asset price. For example:

- buying one share of an asset priced at £10 requires £10 – a small amount;
- buying one share of an asset priced at £1,000 requires £1,000 – a large amount.

Although both actions would be described as buying one share, they represent very different levels of investment.

To avoid this problem, the amount involved in each transaction is fixed relative to the initial capital. In this study, one transaction represents 10% of the initial capital:

$$\Delta = 0.1C_0$$

Therefore, a Buy Action=increase exposure by $0.1C_0$ regardless of the current asset price.

The corresponding number of units purchased is determined by:

$$\Delta h_t = \frac{0.1C_0}{P_t}$$

This means the monetary size to open any position remains consistent throughout the trading period, while the number of units purchased changes with the asset price.

This makes trading actions easier to compare across different periods in the dataset. Changes in the asset price do not change the intended size of the agent's decision, allowing the analysis to focus more directly on the trading policy learned by the reinforcement learning algorithms.

3.4.4 Action Constraints

Although the agent has three possible actions, some actions may not be possible at a particular timestep because of the current portfolio position.

For example:

- the agent cannot buy if it does not have enough available cash;
- the agent cannot sell if it does not hold enough of the asset to withdraw.

To handle these situations, the environment uses a legal action mask. The mask does not provide any additional market information to the agent. It simply indicates which actions can be carried out given the current portfolio status.

The mask is represented as:

$$M_t = \left[m_{\text{hold}}, m_{\text{buy}}, m_{\text{sell}}\right]$$

where each value indicates whether the corresponding action is currently allowed.

The availability of each action is determined from the portfolio state:

- available cash determines whether a buy action can be carried out;
- the current asset holding determines whether a sell action can be carried out;
- hold remains available throughout the episode.

Therefore, the action mask can be represented this way:

$$M_t \in \{0,1\}^3,$$

where $M_t(a) = 1$ indicates that action (a) is available and

$M_t(a) = 0$ indicates that it is not.

Because the mask is derived directly from information already in the portfolio state, it is not treated as an additional feature of the state representation. Instead, it acts as a constraint on the agent, preventing it from selecting actions the environment cannot execute.

3.4.5 Transaction Costs

A transaction fee is charged whenever the agent carries out a buy or sell action. The fee is set at:

$$c = 0.05\%$$

For a transaction of value Δ, the fee is:

$$Cost = \Delta c$$

The same fee calculation is applied to both purchases and sales.

Including transaction costs matters because it prevents the agent from overly buying and selling repeatedly. Without these fees, the agent could benefit from making many trades even when price movements are very small, making frequent trading appear more profitable than it would be in practice.

The inclusion of fees therefore introduces a basic cost of trading into the environment and provides a more realistic basis for evaluating learned trading strategies.

3.4.6 Relationship Between the State and Actions

The state representation and the action space are closely related. For the agent to make a meaningful action, the state must provide enough information about the current situation.

For example, deciding whether to buy an asset may depend on several factors, including:

- the agent's current exposure to the asset;
- the amount of cash available;
- whether an existing position is making a profit or loss;
- how much time remains in the trading period;
- the current market conditions.

The same applies to a sell. Selling may be appropriate for different reasons, such as taking a profit, reducing a loss, or responding to a change in market conditions.

The final action space was therefore designed to intentionally match the state representation written in Section 3.3. The state provides the information needed to make an informed decision, while the action determines what the agent does.

The agent decides whether to hold, buy, or sell when appropriate, while the environment determines the amount traded and prevents invalid actions that cannot be carried out.

3.5 Reward Function Design

3.5.1 Purpose of the Reward Function

The reward function tells the reinforcement learning algorithm how well it performed after taking an action and guides the agent toward more desirable actions. Unlike supervised learning, where the model is given the correct answer, the agent learns by taking actions and receiving feedback on the results of those actions.

In a trading environment, the reward function needs to accurately reflect the actual objective of the investment strategy. If it is poorly designed, the agent may learn behaviours that are not desirable, such as trading too frequently, taking unnecessary risks, or performing well during training but poorly under realistic conditions.

The main goal of this dissertation is to investigate whether reinforcement learning algorithms can learn profitable trading strategies for the selected asset while accounting for transaction costs. For this reason, the primary reward is based on the change in portfolio wealth from one timestep (trading day) to the next.

3.5.2 Wealth-Based Reward

The main reward used in this study is based on the change in the portfolio's total value:

$$r_t = W_{t+1} - W_t$$

where portfolio wealth at timestep t is given by:

$$W_t = C_t + h_t P_t$$

Here:

- C_t represents the cash available in the portfolio;
- h_t represents the number of asset units held;
- P_t represents the current asset price.

The reward therefore measures how much the portfolio's total value changes from one timestep (trading day) to the next after the agent's action.

A positive reward means the portfolio increased in value, while a negative reward means it decreased. This gives the agent a direct measure of the financial outcome of its decisions.

3.5.3 Including Transaction Costs in the Reward

Transaction fees are included when calculating the reward so that the result reflects the actual cost of carrying out a trade.

When a buy or sell action is taken at timestep t, the portfolio value is updated after the transaction fee is deducted:

$$W_{t+1} = C_{t+1} + h_{t+1}P_{t+1} - Fees$$

The reward is then calculated from the resulting change in portfolio wealth. This means that the agent is rewarded or penalised based on the value of the portfolio after trading costs have been taken into account.

3.5.4 Wealth Change As the Primary Reward

The change in portfolio wealth was chosen as the main reward for several reasons.

First, it directly reflects the main objective of the trading strategy: increasing the value of the portfolio.

Second, the reward is straightforward to interpret. A positive value means that the portfolio has gained value, while a negative value means that it has lost value. This also makes the reward closely related to the metric used to evaluate investment performance in the real world.

Third, it satisfies the requirements of the MDP defined in Section 3.2 because the reward can be obtained from portfolio information in the state representation.

The current portfolio wealth can be recovered from the cash and exposure features in the state representation:

$$c_t + v_t = \frac{W_t}{C_0}$$

Therefore, the change in wealth can be written as:

$$W_t = C_0 \left[(c_t + v_t)\right]$$

$$r_t = C_0 \left[(c_{t+1} + v_{t+1}) - (c_t + v_t)\right]$$

This shows that the reward depends only on the observed transition:

$$(s_t, a_t, s_{t+1})$$

without requiring additional information unavailable to the agent.

3.5.5 Risk-Aware Reward Extension

Although the main objective is to increase portfolio wealth, focusing only on returns may encourage the agent to take more risk than desired. For example, a strategy may achieve a high final return but experience large losses along the way, which is not a desirable trading behaviour.

Therefore, a risk-adjusted reward that accounts for changes in portfolio drawdown is considered as an additional experimental condition:

$$r_t^{\text{risk}} = \Delta W_t - \lambda \, C_0 \, \Delta dd_t$$

where:

- ΔW_t is the change in portfolio wealth;
- Δdd_t is the increase in portfolio drawdown at step t (and is zero when drawdown does not rise);
- C_0 is the initial capital;
- λ controls the penalty applied to increases in drawdown.

Drawdown measures how far the current portfolio wealth has fallen from the highest level reached so far:

$$dd_t = \frac{\max_{u \le t} W_u - W_t}{\max_{u \le t} W_u}$$

where W_t is the portfolio wealth at time t, and W_u is the wealth at any earlier time u up to t. $\max_{u \le t} W_u$ is therefore the highest wealth reached so far in the episode.

Δdd_t is therefore the increase in drawdown from one trading day to the next. Multiplying it by the initial capital C_0 keeps the reward in dollars, so a penalty of $\lambda = 0$ lets the wealth-change reward remain the same, while a larger λ discourages actions that worsen the drawdown.

However, this reward requires access to the portfolio's previous performance because the previous highest wealth reached up to time t must be known in order to calculate drawdown.

For this reason, the risk-aware reward is used only with the risk-aware state extension described in Section 3.3.6. This ensures that the information needed to calculate the reward is available to the agent.

The wealth-change reward remains the main reward used in the baseline experiments. The risk-aware version is treated separately so that the effect of explicitly including information about portfolio loss history can be examined.

3.5.6 Limitations of the Reward Function

Although the wealth-change reward provides a clear and direct measure of trading performance, it does not capture every aspect of a desirable trading strategy.

For Example, it does not directly penalise:

- large falls in portfolio value;
- unstable performance in returns;

- high levels of volatility.

A strategy could therefore achieve a strong overall return while experiencing substantial losses during the trading period. The limitations are addressed by the risk-aware reward in Section 3.5.5 by adding a penalty when drawdown increases.

However, adding more objectives changes the learning problem and what the agent is being trained to optimise. This can make comparisons between algorithms less straightforward because improvements may result from the different reward definition rather than from the learning algorithm itself.

Therefore, the wealth-change reward is used as the common baseline, while the risk-aware reward is considered separately as an additional experiment.

3.6 Objective Function

The objective function describes what the reinforcement learning algorithms are trying to achieve during training. In this study, the aim is to learn a trading policy that produces the highest possible expected return over each trading episode.

The reward function introduced in Section 3.5 measures the result of an individual decision, while the objective function extends this idea by combining the rewards received throughout the episode.

3.6.1 Episode Return

An episode/trajectory can be represented by the sequence:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$$

where s_t is the state observed at timestep t,

$a_t$is the action selected by the agent, and

r_t is the reward resulting from that action.

The episode contains T trading decisions and ends when it reaches the final market observation. Any position that remains open at this point is then closed according to the rules of the trading environment.

The total discounted return from the beginning of the episode is:

$$G_0 = \sum_{t=0}^{T-1} \gamma^t r_t,$$

Where $\gamma \in (0, 1]$ is the discount factor.

The same calculation can be made from any point within the episode. The return from timestep t, commonly referred to as the reward-to-go, is:

More generally, the return from time t, known as the reward-to-go, is

$$G_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}.$$

This represents the rewards expected from the current timestep onwards. It is particularly relevant to the REINFORCE implementation because it associates an action with the rewards that follow it, rather than assigning the episode's total return to every action taken in that episode.

A discount factor of

$$\gamma = 0.99$$

is used throughout the experiments. The trading episodes contain approximately 18–23 daily observations, so this value results in relatively mild discounting over the length of an episode. For example:

$$0.99^{20} \approx 0.82.$$

Therefore, a reward received near the end of the monthly episode retains most of its original value; ~20 timesteps later, it still contributes around 82% of its original value to the discounted return.

Using $\gamma = 0.99$, therefore, maintains the standard reinforcement-learning formulation for discounted returns while still ensuring the objective is maximising end-of-month wealth.

3.6.2 Expected Return

The outcome of a single trading episode is not enough to judge the quality of a policy because each episode can reflect different market conditions experienced during that month. For instance, a policy that performs well in one month may not perform equally well in another. Therefore, the objective is defined using the expected returns the policy earns across different trading episodes.

The policy is represented by

$$\pi_\theta(a|s),$$

where θ represents the trainable parameters of the policy network. For a given state, the policy assigns probabilities to the available actions, allowing the agent to choose between Buy, Hold, and Sell.

$$\pi_\theta(a_t \mid s_t) = \begin{bmatrix} P(a_t = \text{Hold} \mid s_t) \\ P(a_t = \text{Buy} \mid s_t) \\ P(a_t = \text{Sell} \mid s_t) \end{bmatrix}$$

The expected return is defined as:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[G_0(\tau)]$$

The learning objective is therefore:

$$\theta^* = \arg\max_\theta J(\theta).$$

In simple terms, the aim is not for the agent to find a policy that performs well in only one trading period. Instead, the agent should learn policy parameters that produce good, expected performance across the different monthly market episodes used during training.

The policy parameters indirectly affect the objective of maximizing returns through the actions the agent selects. Changing θ changes the probabilities of choosing Buy, Hold or Sell in each state. These decisions then affect the portfolio over the episode's monthly period and, in turn, the rewards collected throughout.

The reward function itself does not depend on θ. The trading environment, including portfolio wealth, transaction costs, and the agent's selected action, determines it. From this, the learning algorithm determines the policy, while the environment determines the outcome of that policy.

3.6.3 Objective and the Trading Environment

The formulation above also highlights the difference between the trading environment's role and the learning algorithm's role.

The environment determines:

- the market data used by the agent;
- the current portfolio state;
- the transaction costs;
- which actions are currently valid and permitted;
- the resulting portfolio wealth;
- the reward received by the agent; and
- whether the trading episode has ended.

The learning algorithm, on the other hand, determines how the agent updates the policy or value function using its experience.

This separation is important for comparing the different learning approaches in this dissertation. The same trading environment can be used for all the different reinforcement learning algorithms without changing the conditions they operate under. Therefore, REINFORCE, DQN, and PPO use the same state representation, action space, market data, and reward function. Differences in their results would therefore stem from how they learn from the same trading environment.

3.7 REINFORCE: Monte-Carlo Policy Gradient

REINFORCE is the first policy-gradient method used in this study. Unlike value-based methods, which estimate how useful different actions are in a given state, REINFORCE directly learns a policy that gives probabilities to the available actions.

It provides a useful baseline because its main idea is straightforward. The agent selects actions according to its current policy, observes the resulting rewards, and then adjusts the policy so actions can lead to better returns in the future.

3.7.1 Policy Representation

The policy is represented by a neural network with trainable parameters θ. Given the current state s_t, the network produces one output, or logit, for each of the three possible actions:

$$\mathcal{A} = \{\text{Buy},\ \text{Hold},\ \text{Sell}\}.$$

Actions that are not permitted at the current timestep using the action mask described in Section 3.4 are removed. The remaining logits are then passed through a SoftMax function to produce the policy probabilities:

$$\pi_\theta(a_t|s_t).$$

During training, the agent samples an action from this probability distribution. This lets it try different actions and explore alternative trading decisions rather than always choosing the action with the highest probability.

During evaluation, the agent selects the valid action with the highest probability without exploring.

The policy network used in this study has two hidden layers, each with 128 units, and uses the hyperbolic tangent tanh as the activation function. The final layer produces one output for each of the three available actions.

3.7.2 Deriving the Policy-Gradient Objective

The objective introduced in Section 3.6 is an expected return over trading episodes:

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}[G_0(\tau)].$$

To improve the policy, the learning algorithm needs to determine how a small change to the policy parameters (θ) affects this expected return. This requires calculating the gradient:

$$\nabla_\theta J(\theta).$$

The difficulty is that the probability of a complete trading episode $p_\theta(\tau)$ depends on the initial state, the actions selected by the policy, and the resulting environment transitions $(s_0, a_0, r_1, s_1, a_1, r_2, ..., s_T)$. It can be written as:

$$p_\theta(\tau) = \rho(s_0)\prod_{t=0}^{T-1}\pi_\theta(a_t|s_t)\,P(s_{t+1}|s_t, a_t),$$

where $\rho(s_0)$ represents the probability distribution of initial states and

$P(s_{t+1}|s_t, a_t)$, represents the environment's transition to the next state s_{t+1} given the current state and action (s_t, a_t).

The important point is that the policy parameters (θ) affect the action probabilities $\pi_\theta(a_t|s_t)$ but not the market transition process. REINFORCE therefore does not need to model this market transition explicitly, just the action probabilities.

So, we use the likelihood-ratio identity, which provides a way of expressing the gradient $\nabla_\theta J(\theta)$ using the probability of the sampled episodes $p_\theta(\tau)$:

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\nabla_\theta \log p_\theta(\tau).$$

The policy gradient can then be expressed as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\nabla_\theta \log p_\theta(\tau)\, G_0(\tau)\right].$$

Since only the policy terms depend on (θ), this simplifies to

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta\,(a_t|s_t)\, G(\tau)\right].$$

This is the basic policy-gradient expression REINFORCE uses in this study.

In simple terms, the equation adjusts the probability of an action based on the outcome that follows it. Actions resulting in higher returns are encouraged by increasing their probability, while actions associated with lower returns are made less likely.

3.7.3 Reward-to-Go

Using the complete episode return $G(\tau)$ for every action is inefficient because an action taken at time t cannot influence rewards already received before that time t.

For this reason, REINFORCE uses the reward-to-go:

$$G_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}.$$

This works with the sequence of decisions made during the trading episode. Therefore, only rewards from time t onwards are used when evaluating the outcome associated with that decision.

The policy gradient can therefore be written as:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta\,(a_t|s_t)\, G_t\right].$$

where:

$\nabla_\theta \log \pi_\theta\,(a_t|s_t)$ represents the direction in which the policy's action probability changes;

G_t represents the return obtained after taking action (a_t)

This formulation links each action more directly to one another and to the rewards that could have resulted from that action.

3.7.4 Monte-Carlo Estimation

The expectation $\mathbb{E}$ in the policy-gradient equation cannot normally be calculated because the full distribution of possible market trajectories is unknown. REINFORCE therefore uses a Monte Carlo approach: the agent interacts with the trading environment and uses the resulting episodes to estimate the gradient.

For N sampled episodes, the gradient can be approximated as:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{n=1}^{N} \sum_{t=0}^{T_n-1} \nabla_\theta \log \pi_\theta \left(a_t^{(n)} | s_t^{(n)} \right) G_t^{(n)}.$$

The superscript n represents the number of the sampled episode.

The gradient is therefore estimated from the average experience collected from the sampled episodes.

This is why REINFORCE is described as a Monte-Carlo policy-gradient method: The method does not require the agent to know in advance how the market will move or to build a separate model of future price changes. Instead, it learns how to estimate the expected policy gradient from complete sampled episodes produced by its interaction with the environment.

3.7.5 Loss Function Used for Training

Most neural-network frameworks are designed to minimise a loss function, while reinforcement learning is to maximise expected rewards $J(\theta)$. The policy-gradient objective can therefore be written as a loss by introducing a negative sign:

$$L(\theta) = -\frac{1}{N} \sum_{n=1}^{N} \sum_{t=0}^{T_n-1} \log \pi_\theta \left(a_t^{(n)} | s_t^{(n)} \right) G_t^{(n)}.$$

The negative sign changes the original maximisation problem into a minimisation problem that can be handled by the PyTorch optimiser.

The logarithm appears as a direct result of the likelihood-ratio derivation above; it is not an additional trading assumption.

The implementation uses the Adam optimiser [6] with a learning rate of:

$$10^{-3}.$$

During implementation, the sampled returns may also be normalised before being used in the update. This helps control the gradient magnitude across different training batches and can improve numerical stability.

3.7.6 Strengths and Limitations

REINFORCE has several advantages that make it a suitable baseline for comparing with other reinforcement learning algorithms in this project. Its mathematical formulation is relatively straightforward; it directly learns the policy and does not require a separate model to develop the market.

Because the method is relatively simple, performance differences can be useful when evaluating more advanced algorithms.

However, REINFORCE also has a main limitation: its gradient estimates for policy updates can have high variance because they depend completely on episode returns. The policy is updated using returns from sampled episodes, and different episodes can produce very different outcomes even in similar trading situations. As a result, the direction and size of the updates can vary considerably between training episodes.

This can make learning less stable, particularly when the available training data is limited. Using reward-to-go and standardising returns can reduce some of this variation, but they do not remove the algorithm's underlying high-variance nature.

For this reason, this provides motivation for comparing REINFORCE with the other methods, DQN and PPO, to see whether they can achieve a more stable or effective learning in the same trading environment.

3.8 Deep Q-Learning

Deep Q-learning provides a value-based alternative to REINFORCE. Rather than directly learning the probability of selecting each action, it estimates the value of each possible action in the current state. This comparison is useful because both methods address the same portfolio trading problem but learn in different ways.

3.8.1 Action-Value Function

The action-value function, commonly referred to as the Q-function, is defined as $Q^{\pi}(s, a)$:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} r_{t+k}\right].$$

This represents the expected future return from taking action a in state s and then continuing to follow policy π.

The optimal action-value function is written as $Q^{*}(s_t, a_t)$. It represents the highest expected return that can be achieved from a given state when the best available actions are selected

For the optimal action-value function, the Bellman optimality equation [1] is:

$$Q^{*}(s_t, a_t) = \mathbb{E}\left[r_t + \gamma \max_{a'} Q^{*}(s_{t+1}, a')\right].$$

In simple terms, the value of an action depends on two things:

1. the reward received immediately after taking the action; and
2. the value of the opportunities available afterwards.

This makes the Q-function a useful representation for the portfolio trading problem because the agent has a small action space.

In practice, the optimal Q-function is approximated by a neural network. Once the values of the available actions have been estimated, the agent selects the action with the highest Q-value:

$$a_t^{*} = \arg \max_{a \in \mathcal{A}_{\text{legal}}(s_t)} Q^{*}(s_t, a).$$

Only legal actions are considered when making this choice. This is important in the trading environment because, for example, the agent cannot buy when it does not have enough cash or sell when it does not hold enough of the asset.

3.8.2 From Q-Learning to Deep Q-Learning

Traditional tabular Q-learning stores a separate value for every state-action pair in a table. This approach is not suitable for the present portfolio trading problem here because the state contains continuous variables and several different market and portfolio features.

For example, volatility and portfolio exposure can take many different values. Converting all nine state features into discrete categories would first require discretising the continuous variables and then creating a very large state space that would also lose information by grouping different values together.

Deep Q-learning avoids this problem by replacing the table with a neural network:

$$Q_\phi(s, a),$$

Where ϕ represents the trainable parameters of the network.

The network takes the nine-dimensional state $s_t \in \mathbb{R}^{9\cdot}$ as input and estimates the Q-value for each available action. These values are then used to determine the most valuable action in the current state.

$$a_t = \arg \max_{a \in \mathcal{A}_{\text{legal}}(s_t)} Q_\phi(s_t, a).$$

The DQN used in the experiments has two hidden layers with 128 units each and uses ReLU activation functions.

3.8.3 Experience Replay

Successive decisions within a trading episode are naturally related because they come from consecutive trading days. If the network were trained directly on these transitions in their original order, it could produce highly correlated consecutive updates.

DQN addresses this issue using an experience replay buffer [7, 5]. Each interaction with the environment is stored as a transition in a memory buffer:

$$(s_t, a_t, r_t, s_{t+1}, M_{t+1}, d_t),$$

where:

- s_t is the current state;
- a_t is the action selected by the agent;
- $r_t$is the reward received;
- $s_{t+1}$is the next state;
- $M_{t+1}$is the action mask for the next state;

- $d_t$indicates whether the episode has ended.

During training, the agent samples random minibatches of previously stored transitions from the replay buffer so the network does not rely only on the most recent transition.

The replay buffer has a capacity of 50,000 transitions, with 64 transitions sampled per minibatch.

This lets the network learn from reusable experiences collected at different points during training and reduces the dependence between consecutive training samples.

3.8.4 Target Network

Using the same network to calculate both the current Q-values and the target values can make training unstable. This is because the network parameters change during training, which also causes the target used for learning to change.

To reduce this problem, DQN uses a second network known as the target network [5]:

$$Q_{\phi^-}(s, a).$$

The target network is kept separate from the main, or online, Q-network. Rather than updating it after every gradient optimisation step, its parameters are periodically copied from the main Q-network.

Keeping a fixed number of updates for the target provides a more stable reference for calculating the temporal-difference target and reduces how rapidly the learning target changes while the main network is being updated.

In this implementation, the target network is synchronised with the online network every 500 training steps.

3.8.5 Temporal-Difference Learning

Unlike REINFORCE, which waits until the end of an episode to calculate the return, DQN updates its estimates using a one-step temporal-difference target.

For a transition, (s_t, a_t, r_t, s_{t+1}), the target Q-value y_t is calculated from the immediate reward and the highest estimated value of the best valid action in the next state:

$$y_t = r_t + \gamma \max_{a' \in \mathcal{A}_{\text{legal}}(s_{t+1})} Q_{\phi^-}(s_{t+1}, a').$$

Action masking is applied here because the agent should not assign value to actions it cannot take in the next state.

When the episode ends, there are no future trading decisions or rewards to consider. The target must therefore account for this condition:

$$y_t = r_t + \gamma (1 - d_t) \max_{a' \in \mathcal{A}_{\text{legal}}(s_{t+1})} Q_{\phi^-}(s_{t+1}, a').$$

Where d_t indicates whether the episode has ended.

When $d_t = 1$, the second term $(1 - d_t)$ becomes zero, leaving:

$$y_t = r_t.$$

This is particularly important in the trading environment because any remaining position is closed at the end of each monthly episode. The agent cannot take another trading action after this point. Including a future Q-value beyond this state would incorrectly treat the episode as if further decisions were possible.

3.8.6 DQN Loss

The online Q-network is trained by reducing the difference between its predicted Q-value and the temporal-difference target.

For a minibatch of size B containing transitions, the loss is defined as:

$$L(\phi) = \left(Q_\phi\left(s_t, a_t\right) - y_t\right)^2$$

$$L(\phi) = \frac{1}{B}\sum_{b=1}^{B}\left[Q_\phi\left(s_b, a_b\right) - y_b\right]^{2.}$$

This is the mean squared error (MSE) between the network's predicted value and the target value.

The main difference from REINFORCE is how it obtains the learning signal. REINFORCE updates the policy using returns calculated from sampled episodes, whereas DQN updates its Q-values using one-step targets that include an estimate of the future value.

3.8.7 Exploration

If the agent always selected the action with the highest current Q-value, it would follow a purely greedy strategy. This can be problematic early in training because the Q-values are initially poor estimates, and the agent may settle on actions before exploring other available alternatives.

To encourage exploration, the DQN uses an $\epsilon - greedy$ strategy

With probability $1 - \epsilon$, the agent selects the valid action with the highest estimated value:

$$a_t = \arg\max_{a \in \mathcal{A}_{\text{legal}}(s_t)} Q_\phi\left(s_t, a\right).$$

With probability ϵ, the agent instead selects an action randomly from the set of valid actions.

The exploration rate starts at:

$$\epsilon = 1.0$$

and is gradually reduced to:

$$\epsilon = 0.05$$

over the first half of training.

This means that early in training, the agent explores the available actions frequently, while later, as training progresses, it increasingly relies on the most valuable Q-values it has learned.

Also, the agent considers only valid actions when exploring. It cannot select an invalid action simply because it is in its exploratory phase.

3.8.8 Strengths and Limitations

DQN has several advantages for the portfolio trading problem considered in this study. Experience replay lets the agent reuse previous transitions collected at different stages of training, while the target network helps to stabilise the learning process. The network also provides separate value estimates for each valid action, making it possible to directly compare the expected value of Buy, Hold, and Sell.

However, DQN also has some limitations. Because the network learns by directly predicting Q-values, its training can be affected by the scale of the reward and target values. This differs from the REINFORCE implementation, where returns are standardised within each episode before the policy update, reducing sensitivity.

Another limitation is that DQN uses bootstrapping. The target partly relies on Q-value estimates given by the network rather than observed rewards. If these estimates are inaccurate, the resulting errors can be carried into later updates and affect future learning.

These differences make DQN a useful comparison to the policy-gradient algorithms used in this study rather than using another implementation of the same learning architecture.

3.9 PPO: Proximal Policy Optimisation

Proximal Policy Optimisation (PPO) [8] is used as the third learning method in this study to provide a more stable policy-gradient algorithm that can be compared with REINFORCE.

REINFORCE updates the policy using the returns obtained from sampled episodes. Although this approach is straightforward, the policy can change considerably between updates, specifically when the sampled returns are large, making training unstable.

PPO reduces this problem by limiting how much the policy changes during each update. It does this by comparing the new policy with the policy from the training data.

3.9.1 Actor-Critic Structure & Advantage Function

PPO uses two neural networks: an actor and a critic.

The actor represents the portfolio trading policy:

$$\pi_\theta\left(a|s\right),$$

where θ represents the parameters of the policy network.

The critic estimates the value of the current state:

$$V_\phi(s),$$

where ϕ represents the parameters of the value network.

The actor selects actions, while the critic estimates the expected future return from a given state. The critic therefore provides a reference to compare the outcome of a policy's action against.

This lets PPO use an advantage value rather than relying directly on the return of a full episode, like in REINFORCE.

The advantage at time t can be represented as

$$A_t = G_t - V_\phi(s_t),$$

The advantage shows whether the selected action performed better or worse than expected for the current state. A positive advantage means the outcome was better than expected for the current state, while a negative advantage means it was worse than expected.

For instance, if the expected return from a state $V_\phi(s_t)$ is relatively low but the selected action a_t produces a much higher return G_t, the advantage will be positive. The policy $\pi_\theta(a|s)$ should therefore increase the likelihood of taking that action in similar situations.

The implementation uses Generalised Advantage Estimation (GAE) to calculate the advantages. Using an advantage estimate helps reduce the variance of policy-gradient updates.

3.9.2 Policy Ratio

PPO measures how much the policy has changed by comparing the new policy with the policy that generated the sampled data.

The probability ratio ρ_t is defined as:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}.$$

ρ_t is used here to differentiate the policy ratio from the trading reward function r_t.

If $\rho_t(\theta) = 1$, the probability of the selected action has not changed from the previous policy.

A ratio greater than 1 means the new policy gives the selected action a higher probability than the previous policy, while a ratio below 1 means its probability has decreased.

Without restrictions, a single update could change the policy substantially. PPO uses the ratio to decide whether a policy update has moved too far from the policy that generated the sampled experience, and it limits that ratio from being used in the policy objective.

3.9.3 Clipped Objective

PPO controls policy updates using a clipped objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)A_t,\ clip\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right)A_t\right)\right].$$

This project uses the standard implementation:

$$\epsilon = 0.2.$$

This gives a clipping range that limits the policy ratio to approximately:

$$[0.8,\ 1.2].$$

Clipping does not prevent the policy from moving outside this range or changing by more than 20%. Instead, once the probability ratio moves beyond the specified range, the objective limits improvement in that direction to prevent the update from becoming even larger.

For example, if an action has a positive advantage, increasing its probability can improve the objective. However, once the change becomes sufficiently large, the clipped objective prevents the update from gaining additional benefit that increases that probability further.

In this way, PPO improves the policy while avoiding large changes between continual updates. This provides a smaller, more controlled learning process than directly updating the policy from sampled returns, like in REINFORCE.

3.9.4 Action Masking in PPO

The action masking described in Section 3.4 also applies to PPO [9]. The agent should not be able to select an action that the trading environment cannot execute.

For this reason, the implementation uses MaskablePPO instead of the standard PPO implementation. The action mask is applied after the policy determines the probability of each action, so it excludes actions that are not currently possible.

For example, when the available cash is less than the amount required to open an asset position, the Buy action is unavailable. Likewise, the Sell action is unavailable when the agent does not hold enough of the asset.

Using the same action mask for REINFORCE, DQN, and PPO ensures that all three algorithms operate under the same trading conditions. This keeps the action-space design in Section 3.4 consistent with all algorithms used in the experiments.

3.9.5 PPO in the Experiment

PPO is included to provide a third comparison with REINFORCE and DQN. The three methods use the same trading environment but learn differently.

Method	Learning approach	Network learns	Main learning signal
REINFORCE	Policy gradient	Policy - $\pi_\theta(a\|s)$	Reward-to-go G_t $$G_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$$
DQN	Value-based	Q-values - $Q_{\phi^-}(s,a)$	Temporal-difference target y_t $$y_t = r_t + \gamma \max_{a' \in \mathcal{A}_{\text{legal}}(s_{t+1})} Q_{\phi^-}(s_{t+1}, a').$$

Method	Learning approach	Network learns	Main learning signal
PPO	Policy gradient with clipped updates	Policy + value function	Advantage Function A_t
		$\pi_\theta(a\|s) + V_\phi(s)$,	$A_t = G_t - V_\phi(s_t)$,

To make this comparison fair and meaningful, PPO uses the same trading environment, state representation, action space & masking, transaction costs, initial capital, data split, and evaluation procedure as the other methods, such that the differences in performance mainly reflect how each algorithm learns rather than differences in the trading environment.

PPO is implemented using MaskablePPO from sb3-contrib [10]. The policy uses the same two-hidden-layer architecture with 128 units per layer as the other neural-network agents. The remaining PPO settings use the library defaults.

3.10 Algorithms

The three learning procedures used in this dissertation are summarised below as implemented. REINFORCE and Deep Q-learning are built from-scratch; MaskablePPO is the library comparator that applies the same action mask.

Algorithm 1: REINFORCE for the trading MDP (as implemented)

1: initialise policy network $\pi\theta$
2: for iteration = 1, 2, ... until step budget exhausted do
3: sample a training month; reset environment (C_0 cash, no position)
4: for $t = 0, \ldots, T-1$ do
5: compute legal-action mask; set illegal logits to $-\infty$
6: sample $a_t \sim \pi\theta(\cdot \mid s_t)$; execute; store $(\log \pi\theta(a_t|s_t), r_t)$
7: end for
8: compute rewards-to-go G_t; standardise within the episode
9: $L(\theta) \leftarrow -\sum_t \log \pi\theta(a_t|s_t)\, G_t$
10: one Adam step on $\nabla\theta L$
11: end for

Notes. $N = 2048$ is the Stable-Baselines3 rollout length used here; $K = 10$ is the number of epochs over each rollout. Month sampling and action masking use the same environment interface as REINFORCE and DQN, so all three algorithms face an identical action space at every step.

The next section states why all three methods are retained in the experimental comparison.

3.11 Comparison of the Three Learning Approaches

REINFORCE and DQN are the primary learning methods because they directly represent the two main reinforcement learning algorithms being investigated: policy-gradient learning and value-

Algorithm 2: Deep Q-learning for the trading MDP (as implemented)

initialise $Q\phi$; target $Q\phi^{-} \leftarrow Q\phi$; replay buffer D
for environment step = 1, 2, ... until step budget exhausted do
with prob. ε: random legal action; else $a_t = \arg\max a$ legal $Q\phi(s_t, a)$
execute a_t; store $(s_t, a_t, r_t, s_{t+1}, \text{mask}_{t+1}, d_t)$ in D
sample minibatch; form masked targets y per (3.16); Adam step on $L(\phi)$
every K steps: $\phi^{-} \leftarrow \phi$
end for

Algorithm 3: MaskablePPO for the trading MDP (as implemented)

initialise actor $\pi\theta$ and critic $V\psi$ (MaskablePPO, MLP 128–128)
for iteration = 1, 2, ... until step budget exhausted do
for step = 1, ..., N do
if episode terminated: sample a training month; reset (C_0 cash, no position)
observe s_t and legal-action mask; sample $a_t \sim \pi\theta(\cdot \mid s_t)$ over legal actions
execute; store $(s_t, a_t, r_t, V\psi(s_t), \log \pi\theta(a_t|s_t))$
end for
estimate advantages $\hat{A}_t$ by GAE; form returns
for epoch = 1, ..., K do
for each minibatch from the rollout do
$\rho_t \leftarrow \pi\theta(a_t|s_t)$ / $\pi\theta\_\text{old}(a_t|s_t)$
Adam step on $L\text{CLIP}(\theta)$ + value loss − entropy bonus
end for
end for
end for

based learning.

PPO is included as an additional policy-gradient comparison to compare the performance of the basic Monte Carlo policy-gradient algorithm with a more stable policy-gradient algorithm.

Using all three algorithms provides a stronger comparison than relying on a single reinforcement learning algorithm. If similar trading behaviour is noticed across the three algorithms, it gives greater confidence that the behaviour is related to the trading environment or state representation rather than a specific algorithm. On the other hand, if the algorithms produce different results, these differences can help show how the choice of learning method affects the resulting trading strategy on the portfolio.

Chapter 4

System Design and Experiment Framework

This chapter explains how the mathematical formulations and design choices from Chapter 3 were implemented in the portfolio trading system using reinforcement learning. The state representation, action space, reward function, and learning algorithms described previously were converted into a working experimental system for evaluation.

4.1 Introduction

The system is a Python-based system, divided into separate components for market data preparation, the trading environment, the learning algorithms, experimental configurations, and the evaluation process. This structure allows easy modification of each component without affecting the rest of the system. More importantly, it allows the same trading environment to be used by different algorithms, REINFORCE, DQN, and PPO, with little difference in their experimental conditions.

REINFORCE and Deep Q-Learning (DQN) were implemented from scratch using PyTorch [11]. PPO was implemented using the MaskablePPO implementation from sb3-contrib [10]. The Gymnasium interface [12] simulates the trading environment and is used across all the learning algorithms.

The implementation also addressed several practical issues stated in the methodology that the mathematical formulation could not capture. These include limiting the agent to only valid actions, ensuring proper separation between training and testing data, closing any remaining position at the end of each monthly episode, and ensuring the risk-aware states and rewards are used separately. This helps ensure that the experiment setup follows the constraints assumed in Chapter 3 to the letter. These details matter heavily because even correct mathematical formulations can produce unreliable results if implemented improperly.

4.2 Data Pipeline

4.2.1 Market Data Preparation

The first stage of the implementation converts historical market data into the observations the simulation trading environment needs. The experiment uses daily OHLCV market data (open, high, low, close, and volume) of the SPDR S&P 500 ETF Trust (SPY) to construct features.

SPY was selected because it is a highly traded ETF that represents the broader US equity market. Using an index ETF rather than an individual company reduces the effect of company-specific events and provides a universal asset for the experiments.

Although most formulations use closing prices, the implementation keeps the open, high, low, close, and volume data. The high and low prices are necessary to calculate the range for the ATR feature used in the state representation.

The main dataset period covers January 2018 to December 2025. This period was stated in Chapter 3 to capture a mix of market conditions while keeping the main experiment within a significant historical period.

The data are obtained using the yfinance interface and are cached locally after processing. This ensures that later experiments use the same prepared dataset rather than retrieving and processing the data again.

4.2.2 Feature Construction & Continuous Calculation

The state features are calculated before separating the data into monthly episodes. This is necessary because some of the features depend on previous data.

For example, momentum uses a five-day window, while the moving average and ATR features use longer windows. Their calculations at the beginning of a month should still use data from previous months and should not restart simply because the environment starts a new monthly episode.

Calculating these features only in each monthly episode would mean that the first few days of every month can't access or use data from the previous month. This would artificially shorten that feature's historical window at the beginning of the experiment.

To avoid this, the pipeline first creates a continuous time series dataset and calculates all features that roll from the past months across the full dataset. It then creates the monthly episodes from the resulting feature data.

More observations are used before the start of the main experiment period to provide the historical data needed to calculate rolling features. For instance, the longest feature window, ATR, uses 20-day periods, so the pipeline includes 21 additional previous day bars before the first actual observation. These past data are used only for calculating the initial features and are not included in the main experiments.

For this reason, the implementation does not begin feature calculation exactly at the first observation of January 2018. Instead, additional market data is loaded before the experimental period begins. This warm-up data provides the previous observations necessary to calculate the first valid features with rolling window calculations.

The final pipeline uses a warm-up period beginning in September 2017. The data from this period are used to initialise the rolling calculations and are then removed before the experimental episodes are created.

This approach avoids using the dataset's first observations with incomplete rolling statistics and treats the first experiment's observation the same as later observations, with the required historical context already available.

The pipeline also checks the processed data for missing values before creating monthly episodes. If the data is not available, the data preparation process raises an error rather than silently producing incomplete features.

Overall, the process ensures that each feature at time t is calculated using information available at or before that time, while still allowing rolling features to use historical observations.

4.2.3 Dataset Split & Monthly Episodes

After feature construction, the continuous dataset is divided into monthly episodes by calendar month. Months containing fewer than 15 trading days are excluded from the dataset. After this

filtering, the dataset is left with 96 monthly episodes.

The experimental data are separated into training, validation, and testing periods. The split is chronological rather than random because the observations form a time series and the aim is to apply the learned policies to current markets.

Figure 4.1 shows the split.

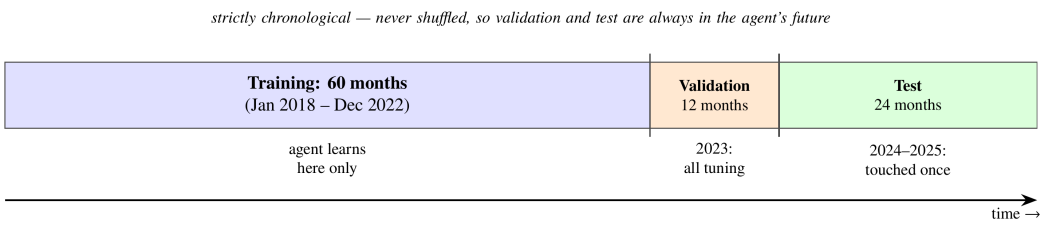

Figure 4.1: Chronological split of the 96 monthly episodes: 60 training months, 12 validation months, and 24 held-out test months.

The split is:

Dataset	Period	Episodes	Purpose
Training	January 2018 – December 2022	60	Learning model parameters
Validation	January 2023 – December 2023	12	Selecting experimental settings
Test	January 2024 – December 2025	24	Final evaluation
Total	2018–2025	96	

The episodes are also kept in chronological order within the splits and are not randomly shuffled between these three periods. The training set contains 60 monthly episodes and is used to learn the reinforcement learning algorithm's parameters. The validation data set contains 12 months and is used to make decisions during development, such as fixed trading amounts. The test set contains the final 24 months and is kept separate from these decisions for only the final evaluation.

Maintaining a chronological split is important for financial data [13]. This prevents observations from later periods from entering the training set and influencing the model's development. This would not reflect how a trading strategy would be developed in practice.

The training period includes most market conditions, including the volatility experienced in 2018, the COVID-19 market crash and recovery in 2020, and the 2022 downtrend. The 2023 period is used for validation, while 2024 and 2025 form the final test period.

The separation between validation and test data is especially important for the experiments in Chapter 5. A configuration that performs well on the validation period may therefore be selected to examine its performance on the held-out test period. The test results are then used to compare the learned strategies.

The processed episodes and their metadata are cached locally so that the same dataset can be used consistently across all experiments.

4.3 Training, Validation and Testing Procedure

The experiments use a fixed sequence that separates training, validation, and test periods. This separation ensures that the test data are not used when making decisions about the model or experimental setup.

4.3.1 Training

During training, each learning algorithm interacts with the monthly episodes in the training period. Each algorithm is given the same market data, state representation, portfolio conditions, and trading constraints. The main difference between the algorithms is how they use the collected experience to update their parameters.

REINFORCE updates the policy using the reward-to-go calculated from completed monthly episodes. DQN updates its Q-network using transitions sampled from the replay buffer. PPO collects fixed-length rollouts and uses the resulting batches for several optimisation epochs.

Each configuration is trained using six random seeds. Using multiple seeds reduces the dependence on using one particular network initialisation or a sequence of sampled experiences and allows the variation between runs to be reported.

The trained model from each seed is saved with its configuration so that it can be evaluated later using the same settings.

4.3.2 Validation

The validation set contains the 12 monthly episodes from 2023.

This period is used for decisions that need to be made before the final test. For example, it determines the fixed transaction amount traded for each Buy or Sell action.

The different calibrations of transaction sizes include:

$$0.05,\ 0.10,\ 0.25,\ 0.50$$

of the initial capital.

The selection is based on the maximum portfolio exposure that can be achieved from each action rather than on trading performance during the test period. This prevents the action from being chosen simply because it happens to perform well on the final evaluation data.

Once the required configuration decisions have been made, the selected settings are fixed for testing.

4.3.3 Testing

The final evaluation uses the held-out data from January 2024 to December 2025.

The test data are not used to make configuration decisions. This ensures that results provide an independent assessment of the methods.

Each trained model is evaluated in the same trading environment and using the same evaluation procedure, which includes the initial capital, transaction costs, action constraints, and monthly episode structure.

For the learned agents, actions are selected intentionally by choosing the highest-probability action for the policy-based methods or the highest-valued action for DQN.

The portfolio outcomes are then recorded for each monthly test episode and each random seed. This produces a consistent set of results that can be used to compare the different learning algorithms.

The stored results therefore contain sufficient information to compare algorithms, state representations, reward functions, and benchmark strategies, which are presented in Chapter 5.

4.4 Experiment Configuration

The implementation was designed so that different experiments could be run by changing their settings without changing the overall trading environment or learning algorithm settings.

For this reason, experiments are defined through configuration files or parameters rather than by manually changing the source code for each run. Each experiment is therefore linked to a specific configuration that records the conditions used to produce it.

This separates experiment settings from the environment's implementation and learning algorithms, allowing the same code to run different state, reward, and action configurations.

4.4.1 Configuration Parameters

The configuration records the main settings that can affect an experiment's outcome. These include:

- learning algorithm;
- state representation and state dimensionality;
- reward formulation;
- transaction fee;
- fixed transaction amounts;
- risk-penalty coefficient, where applicable;
- action model;
- random seed;
- training settings;
- evaluation settings.

Using this approach reduces the need to manually change the source code between experiments. It also makes the experimental process easier to track, since each result can be linked directly to the configuration that produced it.

This is particularly important for the experiments reported in Chapter 5, where several state, reward, and action configurations are compared.

4.4.2 Algorithm Configuration

The experiment uses the same starting capital and transaction cost across all three learning algorithms. The initial capital is set to

$$C_0 = \$10,000$$

and a transaction fee of 0.05% is applied to each trade opened.

The main settings used for REINFORCE, DQN, and PPO are shown below.

Parameter	REINFORCE	DQN	PPO
Network	MLP with 2 hidden layers of 128 $d \to 128 \to 128 \to 3$	MLP with 2 hidden layers of 128 $d \to 128 \to 128 \to 3$	*MlpPolicy* $d \to 128 \to 128$
Activation	tanh	ReLU	
Optimiser	Adam	Adam	Adam
Learning rate	10^{-3}	10^{-3}	$3 * 10^{-4}$
Discount factor γ	0.99	0.99	0.99
Replay buffer	—	50,000	—
Batch size	Episode	64	64
Target network update	—	Every 500 steps	—
Exploration	Policy sampling	$\epsilon - greedy$	Policy sampling
ϵ *schedule*	—	$1.0 \to 0.05$	—
PPO clipping	—	—	0.2
GAE	—	—	0.95
PPO epochs	—	—	10
Entropy coefficient	—	—	0
Training budget	80,000 steps	80,000 steps	80,000 steps
Random seeds	42–47	42–47	42–47

All three methods use the same training budget of 80,000 steps and the same six random seeds to keep the computational budget similar throughout when comparing performance.

The experiments consider the different state and reward configurations described in Chapter 5. These include the standard nine-feature state, the ten-feature state with drawdown, the primary wealth-change reward, the risk-aware reward, and the alternate action configurations.

The implementation also treats each trading position size as a configurable parameter. Its final value is selected using the predefined validation procedure. Chapter 5 reports the different values and their effect on trading results as part of the experimental findings.

4.4.3 Benchmark Implementation

The learned agents are compared with benchmark strategies to provide clear grounds for assessing their performance. These benchmarks are simple trading behaviours that do not require reinforce-

ment learning.

The main benchmark is a buy-and-hold strategy where the initial capital is used to purchase the asset at the start of the episode. The position is then held until the final trading step, when it is closed. The same transaction fee and position closure used for the learned agents apply to this benchmark.

This approach is different from gradually purchasing the asset using the same fixed trade amount available to the learning agents. A gradual purchasing strategy may take several trading days to build a position and, if the fixed trade size is small, it may not become fully invested before the monthly episode ends.

For this reason, the gradual purchase strategy is treated as a separate reference strategy rather than as buy-and-hold. The conventional buy-and-hold strategy gives the learned agents a straight performance target.

The benchmark strategies use the same portfolio conditions, transaction costs, and evaluation procedures as the learned agents. This keeps the comparison consistent and reduces the chance that differences in results stem from inaccuracies in calculating portfolio performance rather than from the strategies themselves.

4.5 Implementation Verifications

Before using the experimental results for analysis, the implementation was tested to ensure that it followed the definitions and constraints established in Chapter 3.

These tests check that the environment, data pipeline, reward calculation, and action constraints behave as intended.

The final implementation includes 21 automated tests covering all major components, all of which pass.

4.5.1 Environment and Portfolio Tests

The environment tests focus on the correctness of the portfolio calculations during trading.

The tests verify that portfolio wealth matches the cash balance and the market value of current holdings. They also check that Buy and Sell actions update the portfolio correctly and that transaction fees are included in all transactions.

The tests further verify that invalid actions are handled correctly and that each monthly episode ends without an open position.

These checks are important because an error in portfolio accounting would affect the reported rewards and performance measures for all three learning algorithms. Such an error could therefore lead to incorrect conclusions even if the learning algorithms were implemented correctly.

4.5.2 Feature and Data Tests

The feature tests check that the state representation follows the intended information structure defined in Chapter 3.

The tests verify that features do not use future information and that rolling features access the necessary historical information when moving from one monthly episode to the next. They also check that the state dimensions and feature ordering match the reported configurations.

The chronological split is tested to ensure that the training, validation, and test periods remain separate.

This testing process identified an important implementation issue. In an earlier version of the pipeline, rolling features were recalculated separately for each monthly episode. As a result, the rolling windows restarted at the beginning of every month rather than continuing from the previous observations, leading to the first set of poor results.

This caused incomplete feature windows in the first days of an episode. In the affected implementation, this issue occurred in approximately 43.1% of episode steps.

This caused incomplete feature windows at episode boundaries, affecting about 43.1% of episode steps in that implementation.

This problem is what led to calculating rolling market features on the continuous dataset before dividing the data into monthly episodes. The corrected pipeline was then tested again to confirm that the required historical information was retained across episode boundaries.

This correction mattered because it affected not only data processing but also how the state was presented to the learning algorithms.

4.5.3 Reward and Risk Tests

The reward tests verify that the primary wealth-change reward corresponds to the change in portfolio wealth with no risk penalty applied.

The risk-aware reward is tested separately to see whether increasing drawdown does not lead to an increase in the reward due to the risk penalty applied.

An additional test checks the relationship between the risk-aware reward and the state representation. A risk-aware reward cannot be paired with a state that does not have the drawdown feature.

The implementation uses this restriction so an inconsistent combination of state and reward settings cannot run accidentally.

These tests therefore ensure that the relationship between state representation and reward design described in Chapter 3 is also maintained in the implemented system.

4.5.4 Action-Masking Tests

The action-masking tests verify that the environment correctly identifies invalid actions that cannot be executed under the current portfolio conditions.

The tests check that the Buy action is unavailable when the portfolio has insufficient cash and that the Sell action is unavailable when the agent does not hold enough of the asset. They also verify the different configurable fixed trading amounts.

A defensive check is also included in the environment so that an invalid action supplied externally is converted to Hold rather than being executed.

Logs monitor these checks during the experiments. After the final experiment, the recorded rate of invalid actions was zero during evaluation, confirming that the action masks accurately prevented the learning algorithms from executing any invalid trades during the testing period.

4.6 Implementation Corrections

4.6.1 Problems Identified During Development

The verification process identified several implementation issues before the final experiments were run.

The main issues involved the feature calculation, benchmark implementation, and the risk-aware reward configuration.

First, the rolling market features using past observations did not initially contain the needed historical context when a new monthly episode began. These meant that some features were calculated using shorter windows than intended.

Second, an earlier version of the buy-and-hold benchmark used the same transaction size as the learning algorithms. This did not represent a fully invested buy-and-hold strategy, where an asset is bought at the beginning of an episode and held untouched till the end of the experiment, and it caused understated exposure and benchmark performance.

Third, the risk-aware reward could initially be used without requiring the drawdown feature in the state. This created a mismatch between the information needed by the reward and the information supplied to the agent because drawdown information was unavailable.

These issues were treated as implementation errors that needed correcting because leaving any of them in place would affect the validity of comparisons between the learning algorithms and benchmark strategies.

4.6.2 Correction and Re-verification

Each identified issue was addressed through a general correction process:

1. the problem was identified through testing or auditing the implementation;
2. its potential effect on the experiment results;
3. the relevant part of the implementation that needed correcting;
4. adding or updating verification tests where necessary after the correction;
5. completely running the verification tests again.

The buy-and-hold issue, for example, shows why this process was necessary. The affected implementation understated the benchmark by approximately $38.95 per month, or 21.6% before correction.

These values describe only the incorrect implementation, which has been discarded, and are reported here only to show the potential impact of the error. They are not presented as findings about the asset's or the market's performance.

After the corrections, the complete automated batch tests were run again, and all 21 tests passed.

The final experiment results in Chapter 5 come only from the corrected implementation, not the earlier incorrect versions.

4.7 Complete Implementation Pipeline

The complete implementation pipeline is a sequence of stages, from collecting historical market data to storing the final evaluation test results.

The first stage loads the SPY OHLCV data, including an initial warm-up period needed to calculate the rolling features. The market features are then calculated on the continuous dataset using only information available up to said observation. Once feature construction is complete, the data are divided chronologically into the training, validation, and test datasets.

The observations are then grouped into monthly episodes. At the start of each episode, the portfolio resets to the initial capital with no asset holdings. At each timestep, the learning algorithm receives the current state and selects one of the valid available actions. The trading environment then executes the action with the transaction fee, updates the portfolio, returns the reward, and moves to the next state.

REINFORCE, DQN, and PPO all interact with the same trading environment. Their main difference is how each algorithm processes the experience to update their model parameters.

After training and validation, the final experiment settings are fixed, and the trained models are evaluated on the held-out 2024-2025 test datasets. The resulting metrics, monthly results, training information, configuration settings, and performance measures are stored together with information about the software environment used to produce them.

The overall implementation can therefore be summarised as:

Market data → warm-up period → feature calculation → chronological split → monthly episodes → trading environment → experimental configuration → learning algorithm → training → validation → fixed configuration → held-out testing → stored results

This pipeline clearly links the methodological decisions defined in Chapter 3, their implementation in the system, and the experiment results presented in Chapter 5.

Figure 4.2 summarises the flow.

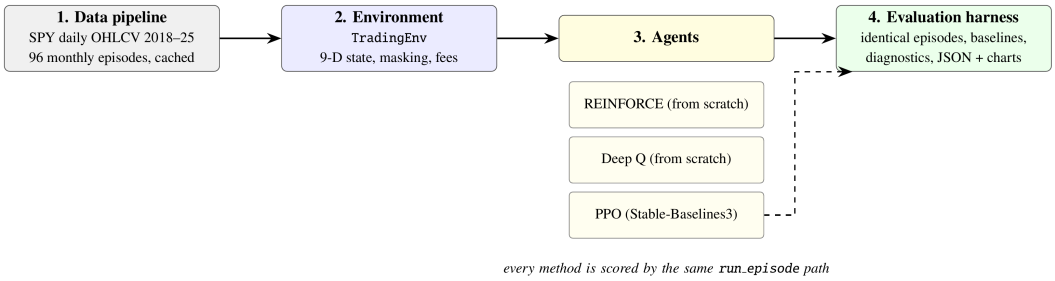

Figure 4.2: System architecture showing the data flow described in Section 4.7. All learned algorithms and benchmark strategies are evaluated using the same episode runner, ensuring that the comparison uses the same evaluation procedure.

Chapter 5

Experimental Results and Analysis

5.1 Introduction

This chapter reports and analyses the experiment results from the trading environment and learning algorithms described in Chapter 4. The experiments examine the financial performance of the learned agents, the effects of two state representations, and their respective reward functions.

The experiments are structured around four questions that directly relate to the objectives of the dissertation to increase portfolio wealth:

1. Performance: Do the learned agents outperform appropriate simple benchmarks on unseen data after accounting for transaction costs?
2. Behaviour: Do the learned agents make use of the market and portfolio information provided in the state representations?
3. State representation: Does adding the drawdown feature to the main nine-feature state affect the learned behaviour or financial performance?
4. Reward: Does the risk-aware reward improve the balance between return and drawdown, rather than simply encouraging the agent to trade less?

A key part of the analysis is determining whether the learned strategies outperform simple strategies. Buy-and-hold, for example, is an important benchmark here. A learned agent producing a positive return does not necessarily indicate that it has learned a useful trading strategy if a simple investment strategy achieves a similar or better result.

For this reason, the analysis considers both financial performance and agent behaviour. Financial performance is evaluated using measures including wealth change, exposure, trading activity, drawdown, and Sharpe ratio. Behavioural analysis examines whether an agent's actions change as the state changes. This helps to identify a policy that responds to state information rather than one that follows a fixed trading pattern.

5.2 Experiment Protocols and Evaluation Criteria

This section describes how the experiments in this chapter were conducted and how the reported results should be interpreted.

5.2.1 Experimental Configurations

Each configuration trained REINFORCE, Deep Q-learning (DQN), and Proximal Policy Optimisation (PPO) separately for 80,000 steps per seed in the portfolio environment,

To account for randomness in reinforcement learning, six random seeds were used, with an initial capital of $C_0 = \$10,000$ at the start of every monthly episode. This means six independent training

runs all with the same configuration, allowing us to analyse the consistency of each algorithm's results rather than relying on a single training run.

During evaluation, all policies acted deterministically: policy-gradient algorithms select the action with the highest probability, while value-based algorithms select the action with the highest estimated value.

The 2023 validation period was used to make every experimental decision reported in this chapter. This included selecting the trade size in Section 5.3, the risk-penalty weight in Section 5.7, and the transaction fee in Section 5.8. These choices were fixed before using them for the main experiments in the 24 held-out test months from 2024–2025.

There are three main learned experiments in this chapter:

- The primary experiment evaluating the nine-feature state with the wealth-change reward.
- The ten-feature state that adds drawdown while keeping the same wealth-change reward.
- The third experiment using the ten-feature state with the risk-aware reward, which actually requires drawdown.

5.2.2 Evaluation Metrics

There are five financial measures used in each experiment report. They are averaged over the episodes and, for learned agents, across the six training seeds:

- Mean wealth change (ΔW) reported in dollars per month;
- Mean exposure (Δv_t), which represents the proportion of initial capital invested in the asset and is averaged across trading days;
- Trades per month;
- Mean episode drawdown (Δdd_t);
- and the monthly Sharpe ratio [14], a financial metric that calculates the average return relative to the variation of those daily returns during the month.

The tables also report the seed spread, which is the difference between the best and worst result across the six seeds. This is included because there might be large differences between training runs which the average alone may hide [15]. A large spread indicates that the algorithm did not produce a consistent trading behaviour across the different runs.

Financial performance is then followed by a behavioural analysis, as returns alone cannot show whether a policy actually used the information in its state. For example, a policy that buys simply because buying is allowed can perform well during a rising market without responding to any market information.

This led to state-dependence analysis, which was used to test whether a policy's actions changed when its observed state changed while the valid actions remained the same. For instance, if a policy selected different actions at different steps with the same actions available, the difference could be attributed to the observed state. The tables report how many of the six seeds met this condition.

5.2.3 Benchmark Strategies

Two non-learning strategies are used as benchmarks. They use the same trading environment, portfolio conditions, and transaction fees as the learned agents.

Buy-and-hold invests the full initial capital on the first trading day and holds the position until the end of the month, when the position is closed. This is the main benchmark as it is a simple passive investment strategy.

Always-buy ($0.25C_0$) purchases a fixed trade amount equal to 25% of the initial capital whenever buying is available, continuing until it has fully invested in that asset. This strategy is useful for comparing against policies that achieve their returns through repeated buying rather than by using information from its observed state, as it also does not use any market or portfolio information when deciding whether to buy.

5.3 Fixed Trade Size Tuning

Before comparing the learned agents with the benchmarks, the fixed transaction size used by the learning agents must be finalized. Determining this trading-action size matters because the amount traded at each decision point determines how quickly the agent can build or reduce its position during a monthly episode.

In the trading environment, each Buy or Sell action changes the trade by a fixed fraction of the initial capital. A smaller value gives the agent finer control over its position but also means that more trading decisions are needed to reach a high level of asset exposure. A larger value lets the agent build its position more quickly but provides less control over the size of each change.

This creates a problem between the learned agents and the buy-and-hold benchmark. Buy-and-hold invests the available capital at the beginning of the episode, while a trading agent must build its position through repeated actions. If the trade size uses a small, fixed transaction size, the agent will need several trading days to reach a similar exposure as buy-and-hold and may not overall reach the same exposure within a typical month. Poor results could therefore arise from the fixed trade amounts rather than from the learned policy.

The trade size was therefore calibrated using the validation data to select the most stable size before running the main comparison between learning algorithms on the held-out data.

5.3.1 Exposure available under different transaction sizes

The validation experiment measures the maximum exposure reachable during a month using a policy that always buys whenever the action is available, for each fixed transaction size. The validation set contains 12 monthly episodes, with an average length of approximately 20.8 trading days. Table 5.1 and Figure 5.1 show the results.

Table 5.1: Attainable exposure for each fixed trade amount on the validation months. The policy buys whenever a purchase is valid.

Trade amount	Buys Days required to purchase a full asset	Mean maximum exposure	Mean wealth change
$0.05C_0$	20	0.450	+$95.49
$0.10C_0$	10	0.690	+$121.79
$0.25C_0$	4	0.838	+$167.39
$0.50C_0$	2	0.886	+$162.77
Buy-and-hold	—	0.910	+$166.92

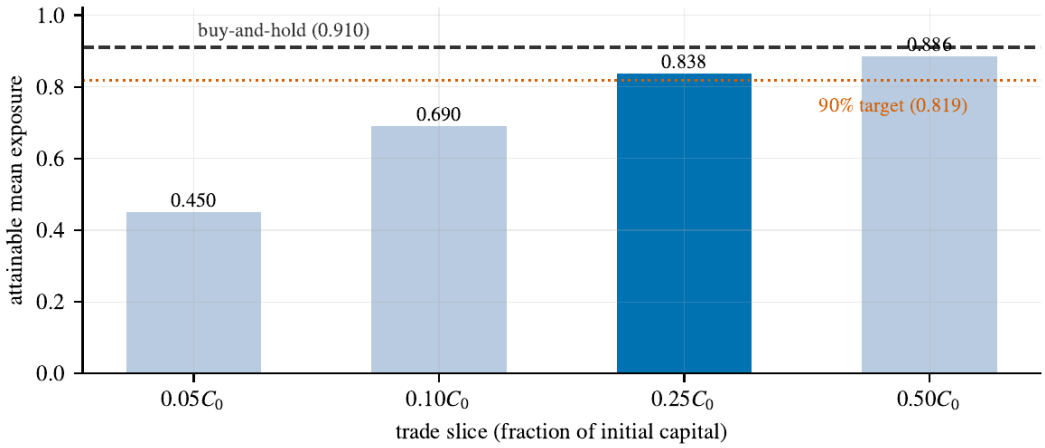

Figure 5.1: Exposure ceilings from Table 5.1. The 0.25 C_0 trade amount (highlighted) is the smallest that clears the 90% target of buy-and-hold's exposure; the 0.10 C_0 amount cannot reach it even in principle.

The results show that the original $0.10C_0$ trade size restricts the agent's asset exposure. This trade size requires ten buy days to fully invest in that asset, but the average episode is only 20.8 trading days long, so half a typical month passes before full investment is even possible. Its mean maximum exposure is 0.690, which is also significantly below the 0.910 asset exposure for the simple buy-and-hold strategy.

Hence, even a policy that buys whenever possible cannot reach the same level of exposure as a simple benchmark during a typical monthly episode. This matters when analysing the agent's performance. A low return may result from poor decision-making, but it may also result from the trading size preventing the agent from investing enough in the asset. The two outcomes should be treated distinctly.

The $0.25C_0$ trade size was therefore selected as the primary configuration, reaching 0.838 exposure, which corresponds to approximately 92% of the buy-and-hold exposure. This selection was based on the criteria that exposure must reach at least 90% of the buy-and-hold, since no trade size can reach that level completely.

The buy-and-hold exposure of 0.910 gives an average exposure of approximately 0.819 under this criterion. The $0.25C_0$ trade size therefore satisfies the requirement as the smallest tested trade size whose exposure is close to the benchmark.

Increasing the trade size setting to $0.50C_0$ raises exposure slightly, from 0.838 to 0.886, but makes each individual action much larger, giving the agent less control over gradual exposure changes.

Based on this calibration, the $0.25C_0$ size balances maximum asset exposure and control over position changes. It gives the learned agents a reasonable opportunity to reach the benchmark level without using a much coarser $0.50C_0$ trade size.

5.4 Do the Learned Policies Use Their Observations?

The Financial performance of the portfolio trading agent on its own does not show whether a learned agent is actually using the information contained in its state representation. For example, an agent that buys whenever buying is possible can perform well in a rising market without using any of the market information its state provides.

This requires examining the behavioural patterns of the agents' learned strategies to identify whether a policy selects a different action when the state changes, provided the available valid actions remain the same. This shows that the policy responds to its observed state rather than simply following action constraints and can be classified as state-dependent.

The experiment is performed on the validation data using the calibrated $0.25C_0$ trade size decided in Section 5.3, with the nine-feature state and the primary wealth-change reward. This test aims only to analyse behavioural differences between the algorithms by determining whether changes in the information provided to the agent are reflected in its decisions. It is not used to determine the final financial performance of the agents.

Every cell was trained from scratch for this study: six seeds per method, 80,000 steps each.

5.4.1 Behaviour of the learned policies

Table 5.2: Behavioural Results on the validation months (0.25 C_0 trade size, nine-feature state, wealth-change reward, six freshly trained seeds). The always-buy strategy shows the maximum asset exposure this trade size can reach.

Policy	Mean ΔW	Seed spread	Exposure	Trades	State-dependent
Always-buy	+$167.39	—	0.838	—	—
REINFORCE	+$154.96	$37.29	0.807	10.3	0/6
Deep Q-learning	+$86.16	$64.20	0.325	8.5	6/6
PPO	+$118.55	$167.39	0.596	3.7	0/6

Table 5.2 summarises the outcome and Figure 5.2 shows every seed individually. The columns read as follows. Mean ΔW is the average wealth change per month, calculated over the 12 validation months and then over the six seeds. Seed spread is the difference between the best and worst seeds; a spread larger than the mean produced noticeably different results and did not agree on a strategy. State-dependent counts how many of the six seeds act according to the experimental behaviour being analysed.

The REINFORCE results are clearer when the individual seeds are read. Four of its six seeds finished at exactly +$167.39 with an exposure of 0.838. This matches the always-buy strategy, which was set as the upper limit. These policies buy on the first four days when buying is valid, hold the position, and close at the end of the month. The other two seeds earned +$130.10 while trading 20.8 times per month: Their actions followed a fixed pattern of trading on each bar rather than changing in response to the market state.

PPO produced three different behaviours across its six seeds. Four seeds reached the always-buy result of +$167.39. One seed never traded at all, producing a wealth change of $0.00 and an exposure of 0.000. The remaining seed earned +$41.73, relying on only two trading styles. This produced a seed spread of $167.39, showing the large difference between the strongest and weakest PPO runs.

Deep Q-learning is the only method where none of its seeds reached either extreme behaviours, unlike the policy-gradient methods. Its results ranged from +$53.23 to +$117.43, with a mean wealth change of +$86.16 and an average exposure of 0.325, less than half the 0.838 standard exposure of the always-buy strategy.

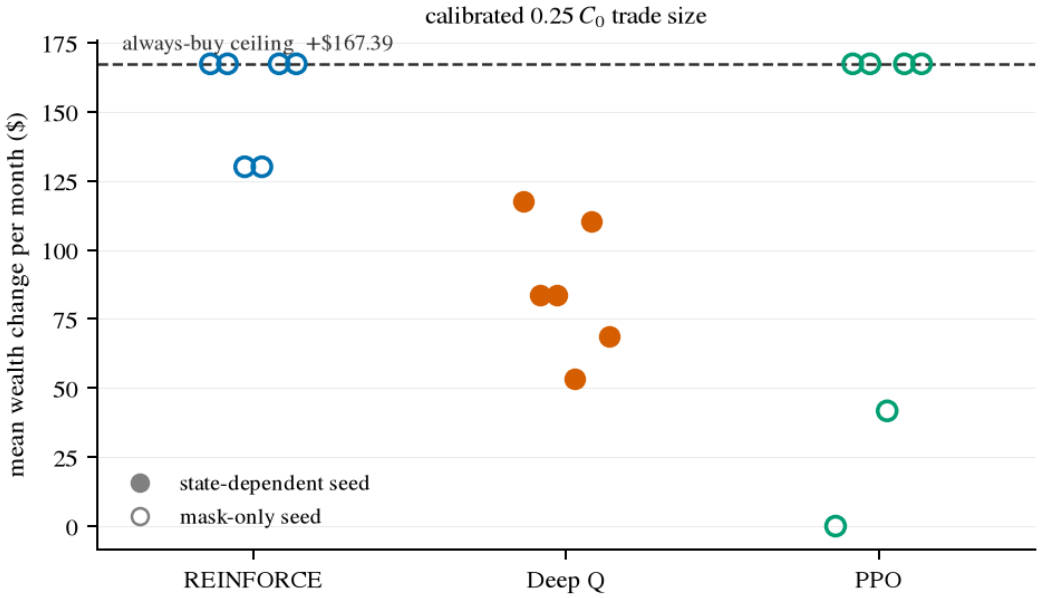

Figure 5.2: Results for each seed from Table 5.2. Hollow-marker seeds show actions that don't depend on their state information; filled-marker seeds are state-dependent ones. Policy-gradient methods (REINFORCE, PPO) produce policies that cluster to either high exposure or collapse towards zero, while all six Deep Q-learning seeds fall between those corners.

5.4.2 State-dependence analysis

The final column in Table 5.2 summarises the state-dependence test, with the count representing the difference in behaviour across the six seeds

For REINFORCE, the result of 0/6 means that none of the six independently trained policies changed their action when the state changed while the same actions remained valid. The four seeds that reached the maximum exposure followed the same basic rule of buying whenever possible, while the other two followed a fixed trading pattern. The diagnostic therefore found no evidence that these policies used market and portfolio features to change their decisions.

Deep Q-learning produced the opposite result. All six seeds were classified as state-dependent. Each seed selected different actions in states where the same valid actions were available. This indicates that the observed current state influences the decisions made by the DQN policies.

PPO produced the same result as REINFORCE, with 0/6 state-dependent seeds. Taken together, the splits are categorical by algorithm family across the three methods: all six DQN seeds were state-dependent, compared with none of the 12 policy-gradient seeds from REINFORCE and PPO.

5.4.3 Interpretation of the learned trading behaviour

The different results across the three methods suggest that the learning approach had a strong influence on the type of trading behaviour that developed. REINFORCE and PPO frequently moved towards simple policies that did not change their actions in response to the observed state, whereas Deep Q-learning showed state-dependent behaviour across all six seeds.

One possible explanation is the difference between policy-gradient and value-based learning. REINFORCE updates the probability of an action according to the return that follows it. When the market generally favours holding an asset, repeatedly buying can produce a strong return without

needing to sample other policies. The learning signal may therefore settle on a simple high-exposure strategy for higher rewards rather than exploring.

PPO uses a different update rule from REINFORCE, but it still learns a policy directly. Its clipped objective prevents large policy changes, which can make learning more stable, but it does not need the policy to become state-dependent. If a simple trading rule produces consistently positive returns, the policy may face little pressure to learn a more complex relationship between state and action. The extreme seed outcomes are what hint at this. Seeds from both policy-gradient methods finish at the always-buy upper limit, since that is the most rewarding.

Deep Q-learning learns differently. Rather than adjusting action probabilities directly, it estimates the value of each available action in the current state and picks the action with the highest estimate at each step. When future estimated values differ, the chosen action changes accordingly, and the agent has to distinguish between these actions. The state-dependent behaviour therefore becomes a structural property of this algorithm rather than an achievement, as it necessarily depends on the changes in the market and portfolio state.

These explanations should be treated as interpretations of observed behaviours rather than as direct evidence of the internal learning process. This experiment was not designed to find out the precise reason the algorithms moved to these different strategies. They are, however, consistent, as different seeds show that learning algorithms affect how available state information is used to develop a trading strategy.

5.5 Effect of the State Representation

Chapter 3 defines the nine-feature state as the main state representation used in this study. It includes market, portfolio, position, and time information. Drawdown is intentionally treated separately as an additional risk-related feature rather than as the last stage of the state design.

This experiment therefore focuses on a specific question: does adding drawdown to the nine-feature state produce a meaningful change in performance or behaviour when the reward function remains unchanged?

The comparison keeps the following conditions fixed:

- The same wealth-change reward;
- the same $0.25C_0$ trade size;
- the same held-out test period;
- the same six random seeds.

The intended difference between the two configurations is therefore only the inclusion of drawdown in the state.

5.5.1 Nine-Feature vs Ten-Feature State

Method	State	ΔW ($)	Spread	Exposure	Trades	DD	Sharpe	State-dep.
REINFORCE	9-feature	+171.51	24.7	0.810	10.3	0.0266	0.675	0/6
REINFORCE	10-feature	+167.39	24.7	0.794	13.0	0.0263	0.670	0/6

Method	State	ΔW ($)	Spread	Exposure	Trades	DD	Sharpe	State-dep.
Deep Q	9-feature	+74.68	71.4	0.312	8.2	0.0124	0.507	6/6
Deep Q	10-feature	+77.81	89.5	0.354	7.4	0.0140	0.505	6/6
PPO	9-feature	+127.35	179.8	0.598	3.7	0.0195	0.562	0/6
PPO	10-feature	+159.97	94.0	0.759	7.3	0.0250	0.671	1/6

Table 5.3: Results of the nine- and ten-feature states on the test data (wealth-change reward, 0.25 C_0 trade size, six seeds). DD shows the mean drawdown within each episode, while State-dep. shows the number of seeds where actions changed with the observed state.

The effect of adding drawdown and the changes in mean wealth performance for REINFORCE and DQN are small.

For REINFORCE, wealth change falls from $171.5 with the nine-feature state to +$167.39 with the ten-feature state. The difference is only −$4.12 per month against a seed spread of $24.70 for both configurations, which is considerably larger than the mean wealth change.

The behavioural result also remains unchanged. None of the six seeds are classified as state-dependent in either configuration. Adding drawdown therefore did not produce a noticeable change in the behaviour of the REINFORCE policies.

DQN shows a small change in the opposite direction. Wealth increases from +$74.68 to +$77.81, a change of +$3.13 per month. The seed spreads are $71.40 and $89.50, respectively, so the change between the two state configurations is again much smaller than the variation across seeds.

The behavioural result is unchanged for DQN. All six seeds remain state-dependent with both state representations.

PPO appears to improve, from +$127.35 to +$159.97, gaining $32.62. However, the result needs to be considered alongside the variation between seeds. The nine-feature configuration has a spread of $179.80—larger than the mean itself — but it narrows to $94.00 in the ten-feature configuration, with only one seed becoming state-dependent.

The change in the mean is therefore not enough, on its own, to show that adding drawdown caused the improvement.

5.5.2 Analysis When Drawdown Was Added

The other performance measures columns tell the same story in different parts. For DQN, average exposure rose from 0.312 to 0.354, and mean drawdown from 0.0124 to 0.0140 after adding that feature to the state — return, exposure, and risk all moved up together by similar proportions. Its Sharpe ratio, however, changes very little, from 0.507 to 0.505.

The patterns noticed here suggest the additional drawdown feature did not lead to more effective risk-aware decisions, but to slightly larger asset exposure

The changes for REINFORCE barely move. Exposure falls from 0.810 to 0.794, while drawdown decreases slightly from 0.0266 to 0.0263, as expected of a policy algorithm that does not read the state it is given.

The small changes in the financial measures therefore do not indicate a change in the basic policy behaviour.

Overall, the results do not show a consistent improvement in either performance or behaviour from adding drawdown to the state while using the default wealth-change reward.

5.5.3 Interpretation

These results do not mean that drawdown information is unimportant. They show that, in this environment, without changing the reward objective, the additional information did not provide a clear advantage— an agent maximising wealth change in general rising markets has little use for its own loss history.

This matters because Chapter 3 introduced drawdown to provide information that could support risk-sensitive decisions. However, this feature becomes relevant only when the agent's learning objective makes larger drawdowns costly while increasing portfolio wealth, which is precisely what the risk-aware reward experiment in Section 5.7 tests.

5.6 Held-Out Test Results

The main performance question of how each learned strategy, in its final configuration, compares with the simple benchmarks on unseen data is evaluated on the 24 held-out months from January 2024 to December 2025.

At this point, the main configuration has been fixed:

- nine-feature state;
- wealth-change reward;
- $0.25C_0$ trade size;
- 0.05% transaction fee;
- six random seeds.

5.6.1 Overall Comparison with the Benchmark Strategies

Policy	ΔW ($)	Spread	Exposure	Trades	DD	Sharpe	State-dep.
Buy-and-hold	+180.25	—	0.913	2.0	0.0310	0.638	—
Always-buy ($0.25C_0$)	+179.75	—	0.841	5.0	0.0274	0.684	—
REINFORCE	+171.51	24.7	0.810	10.3	0.0266	0.675	0/6
Deep Q-learning	+74.68	71.4	0.312	8.2	0.0124	0.507	6/6
PPO	+127.35	179.8	0.598	3.7	0.0195	0.562	0/6

Table 5.4: test results over the 24 held-out months (2024–2025): benchmarks and the three learned agents in their final configuration, six seeds each.

Table 5.4 is the central result of this dissertation.

Two simple strategies are used as benchmarks. Buy-and-hold invests the full capital on the first day and is the benchmark an investor would actually compare against. Always-buy at $0.25C_0$ buys an asset at a quarter fraction of the initial capital. This takes trades with every valid action until it has fully invested in the asset. It represents a strategy that buys unconditionally at the fixed trade size, without requiring observations.

The central result is straightforward: none of the three learned agents outperformed buy-and-hold during the held-out test period.

Buy-and-hold earned +$180.25 per month, equivalent to a 1.8025% increase of the initial capital per monthly episode, and no learned agent reached this. The closest was REINFORCE at +$171.51, which was $8.74 short of buy-and-hold per month; PPO earned +$127.35 and Deep Q-learning +$74.68.

Always buying at $0.25C_0$ earned +$179.75, within $0.50 of buy-and-hold, and it sets the standard a learned agent must reach/beat before it can claim any skill, as matching it needs no market information at all — only buying whenever buying is available.

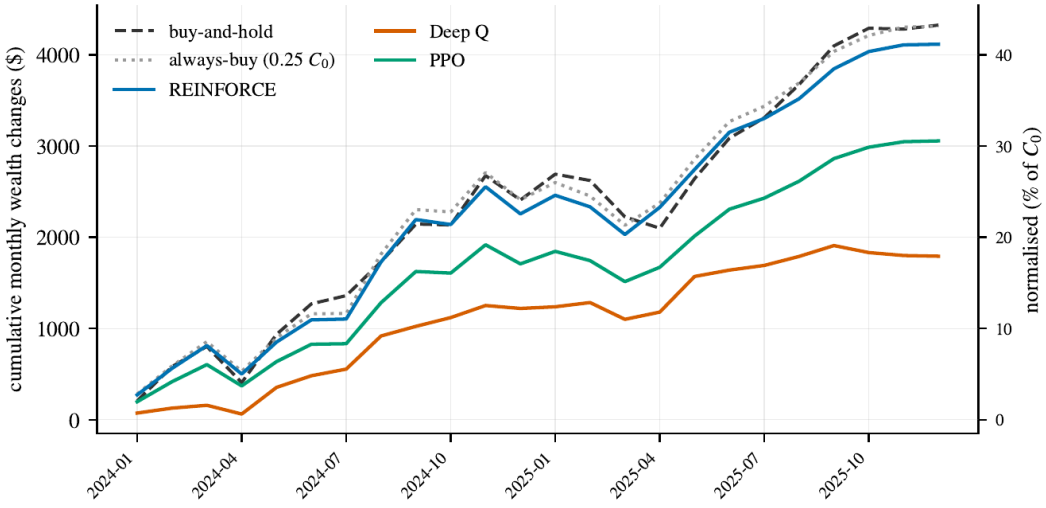

Figure 5.3: Cumulative sum of average monthly wealth changes across the test months. Each monthly episode resets to C_0, so the lines show the sum of monthly gains and losses rather than compound; the right axis expresses percentages of initial capital C_0. REINFORCE follows Always-buy (0.25 C_0) and buy-and-hold almost exactly, while Deep Q-learning shows a flatter line due to low exposure, including through the March–April 2025 decline.

Over the full two years, the gap compounds visibly in Figure 5.3: buy-and-hold accumulates to +$4,326 in monthly wealth change at the end of 2025, REINFORCE to +$4,116 closely following buy-and-hold, PPO to +$3,056, and Deep Q-learning at the lowest value of +$1,792.

5.6.2 Return and market exposure

The exposure column explains most of the return column. The REINFORCE result is particularly interesting; its return was close to buy-and-hold at +$171.51. Its average exposure was 0.810, compared with 0.913 for buy-and-hold, and none of its six seeds were state-dependent, suggesting the high return came mainly from staying invested rather than learning when to enter or leave the market. Its behaviour was therefore closer to the Always-buy at $0.25C$ accumulation strategy. Hence

why it sits within a few percent of Always-buy results (+$179.75, exposure 0.841).

DQN produced a different result. All six seeds were state-dependent, showing that the agent changed its actions in response to state information, and chose to participate the least, with the lowest exposure of 0.312 and a wealth change of $74.68. These results are roughly a third of the buy-and-hold benchmarks, so the lower return may therefore be correlated with reduced market participation.

PPO produced a return between REINFORCE and DQN, with an average wealth change of $127.35 and exposure of 0.598. However, its seed spread was 179.8, which is larger than its mean return. This shows that the six PPO runs produced different outcomes and should not be treated as a single stable trading strategy.

5.6.3 Risk and risk-adjusted performance

Lower returns can still be desirable if they follow a sufficient reduction in risk. For this reason, drawdown and Sharpe ratio are also used alongside return. Figure 5.4 puts every learned configuration in this chapter on that scale.

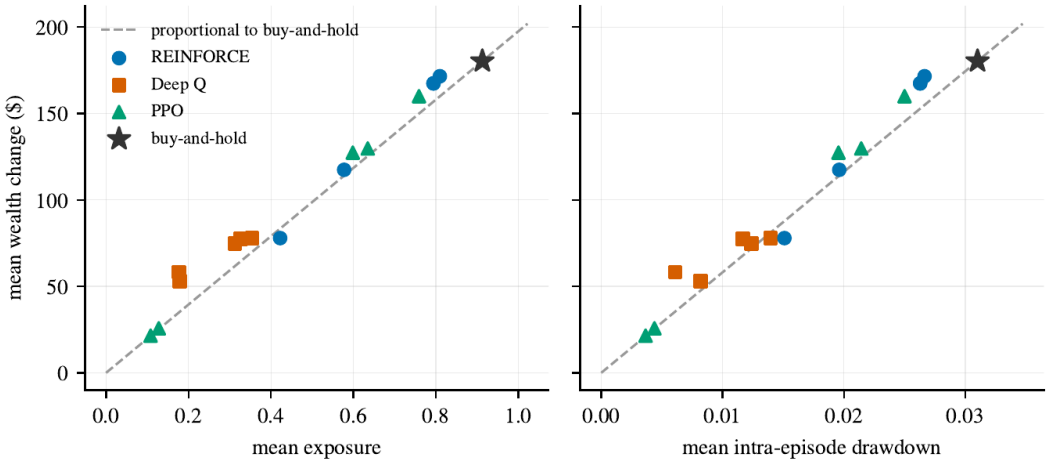

Figure 5.4: Average wealth change compared with exposure (left) and daily drawdown (right) for every learned configuration in this chapter. The dashed lines show the proportional relationship with buy-and-hold. None of the learned configurations show a clear improvement over either line.

Buy-and-hold earned $197 per unit of exposure. An agent that merely holds less of the same asset lands on the dashed proportional line; genuine downside management would appear above it.

Deep Q-learning is an informative case. It earned 41.4% of the buy-and-hold benchmark's return at 34.2% of its exposure (0.312) and 40.0% of its drawdown (0.0124) — return and risk fell equally, which is more like scaling down. Its Sharpe ratio of 0.507 versus buy-and-hold's 0.638 confirms it earned less return per unit of risk exposed to, not more.

The results therefore do not indicate that DQN achieved better risk-adjusted performance than the benchmark. Its lower drawdown largely came from lower exposure, not from a strategy that intentionally avoided the market's decline periods.

5.6.4 Behaviour During the March 2025 Decline

March 2025 provides a useful example because it was the worst test month for the buy-and-hold benchmark. During this month, buy-and-hold lost $398.41, i.e. 3.9841% of the initial capital, with

a drawdown of approximately 5.6%. If the state-dependent agent, Deep Q-learning, had recognised the conditions, this would have been the month to step aside. Instead, DQN had a mean exposure of 0.490 in March— higher than its overall average exposure of 0.312 for the entire test period— and lost $184.04, 46% of the benchmark's loss.

These results are important when interpreting the state-dependence finding. Even in a month when reducing exposure before the decline could have helped, it did not significantly reduce exposure, and these March 2025 losses match its equivalent market participation. Hence, its lower drawdown cannot be attributed to knowing when to avoid bad market periods, but to holding less overall.

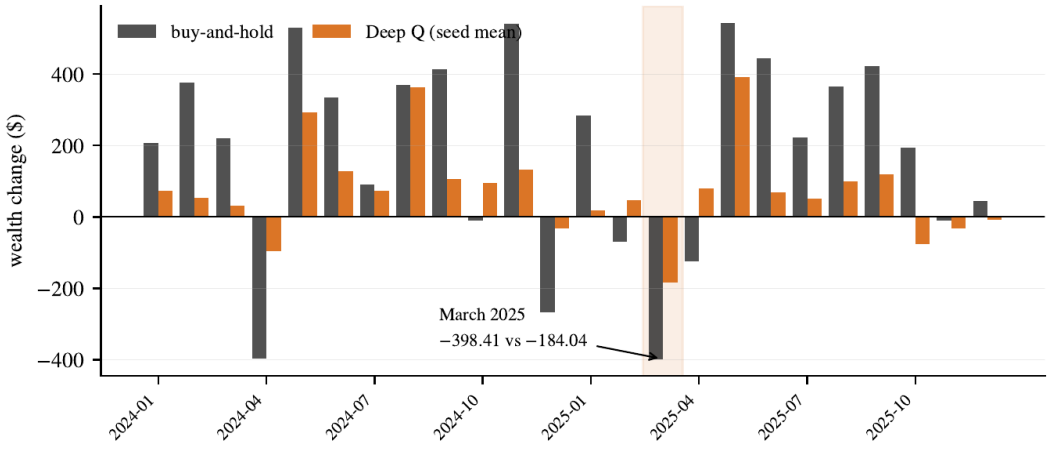

Figure 5.5: Monthly wealth changes of buy-and-hold and Deep Q-learning over the 24 test months. The agent's bars match a 1/3 scaling of the benchmark's bars in almost every month: In March 2025, the worst month, it lost $184.04 compared to the benchmark's $398.41 loss.

Figure 5.5 extends to all 24 months. Each pair of bars is one test month: the dark bar represents buy-and-hold's wealth change, and the orange bar represents the Deep Q-learning for the same month. The two series appear to rise and fall together — the agent's best month is the benchmark's best month (May 2025, +$390.80 compared to +$543.11) and its worst is the benchmark's worst (the discussed March 2025).

Buy-and-hold lost money in seven of the 24 months, and the agent ended positive in three of those seven. Those three months, however, produced little profit compared to the benchmark losses

In the two heavy down months, April 2024 (−$396.08) and March 2025 (−$398.41), the agent was also at a loss—approximately 24% and 46% of the benchmark's loss, respectively—correlating to its 34% average market participation and one-third scaling to the benchmark. Where drawdown management mattered most, the agent still left the position open to losses.

5.6.5 Trading behaviour and Seed consistency

The results also show considerable differences in consistency between the algorithms. To examine this, we analyse seed spreads, which measure differences between runs.

REINFORCE had the smallest seed spread ($24.70), showing seeds agree with each other. However, this consistency should be interpreted with its behavioural results. All six seeds were classified as state-independent and exhibited similar behaviour, precisely because they agree on ignoring market information and generally being highly exposed, as that is more rewarding. The small spread therefore does not necessarily indicate that REINFORCE learned a more reliable trading strategy.

DQN had a larger spread of $71.40. This is consistent with the six seeds learning different state-dependent policies, resulting in greater variations in their returns.

PPO had the largest spread of $179.80, which was greater than the mean return, also showing that individual seeds ranged from near-inactivity to near-full investment in Fig. 5.2, and none were state-dependent.

These results show why evaluating a single random seed would not provide enough evidence for a reliable conclusion about the learning algorithms. Behaviour and performance can vary considerably from one training run to the next.

5.7 Effect of the Risk-Aware Reward

Earlier experiments showed that the main wealth-change reward did not always encourage agents to respond differently to changes in the observed state. It simply rewards changes in portfolio wealth.

Chapter 3.5 identifies the limitation of this objective: it does not penalize large portfolio drawdowns, raising the question of whether explicitly including portfolio drawdown in the reward would lead to more risk-aware behaviour.

To examine this, the risk-aware reward was introduced in which it was modified to include a penalty for increases in drawdown:

$$r_t^{risk} = \Delta W_t - \lambda C_0 \Delta dd_t.$$

where

Δdd_t is the increase in portfolio drawdown at trading day t (and is zero when drawdown does not rise), and coefficient λ controls the penalty effects. The initial capital C_0 factor puts Δdd_t in dollars so that λ is dimensionless.

The risk-aware reward is paired with the ten-feature state using the drawdown feature because it provides the agent with information about its current position relative to its previous maximum wealth.

5.7.1 Selection of the Penalty size

The penalty size was selected using the validation set and Deep Q-learning as the selector because it is the only algorithm whose policies consistently respond to the state (Section 5.4). The held-out test period was not used for this decision.

A selection rule was created. The penalised configuration must keep at least half (50%) of both the unpenalised return—to still earn a decent amount — and the unpenalised exposure—to still participate in the market.

The unpenalised DQN configuration had a mean wealth change of $78.35 and exposure of 0.445. The resulting minimum requirements were therefore:

$$\Delta W \geq \$39.17$$

and

$$Exposure \geq 0.223.$$

The results of the penalty sweep are shown in Table 5.5.

λ	Mean ΔW	Exposure	Drawdown	DD change	Passes
0 (unpenalised)	+$78.35	0.445	0.0163	—	—
0.05	+$88.78	0.401	0.0128	−21.5%	yes
0.10	+$67.30	0.344	0.0124	−24.0%	yes
0.25	+$19.90	0.074	0.0035	−78.4%	no
0.50	+$1.73	0.002	0.0001	−99.4%	no
1.00	$0.00	0.000	0.0000	−100%	no
2.00	$0.00	0.000	0.0000	−100%	no

Table 5.5: Risk-penalty size on the validation months (Deep Q-learning, six seeds). The rule requires ΔW ≥ +$39.17 and exposure ≥ 0.223; the largest passing value that meets both requirements is λ = 0.10.

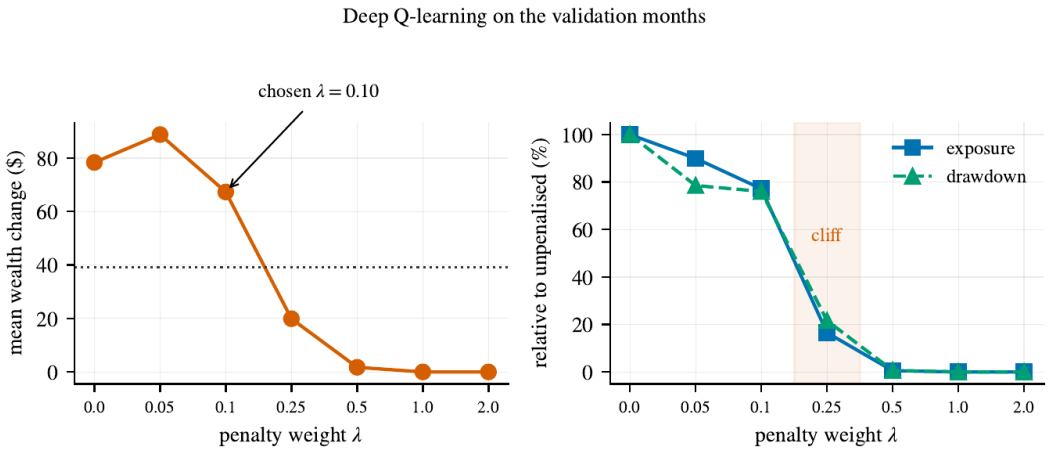

Figure 5.6: Results from the penalty size of Table 5.5. Left: mean wealth change against λ with the requirement floor of +$39.17. Right: exposure and drawdown relative to their values without penalties. Between λ = 0.10 and 0.25, exposure drops sharply rather than decreasing gradually.

The results show a clear change in behaviour of the penalty λ as the size increases. At $\lambda = 0.10$, the agent still maintains a meaningful level of exposure and a return of $67.30.

Increasing the penalty to 0.25 causes exposure to fall sharply to 0.074. The agent loses about 78% of its exposure, indicating reluctance to trade. At $\lambda = 0.50$ and above, the agent stops trading completely.

This result depicts an important limitation of the reward formulation. The simplest way for an agent to reduce drawdown is to hold less of the asset or simply not hold it at all. In theory, a portfolio that remains in cash is not exposed to asset movements and therefore does not experience market drawdowns. Once the drawdown penalty becomes too large, market participation becomes unattractive.

Based on these criteria, the largest coefficient that satisfied both the return and exposure requirements was therefore:

$$\boxed{\lambda = 0.10}.$$

This became the fixed penalty value for evaluating the main risk-aware experiment.

5.7.2 Effect on the Test Set

The selected risk-aware reward was then evaluated on the held-out test set.

Algorithm	Reward	ΔW ($)	Spread	Exposure	Trades	DD	Sharpe
REINFORCE	wealth	+167.39	24.7	0.794	13.0	0.0263	0.670
REINFORCE	risk	+167.39	24.7	0.794	13.0	0.0263	0.670
Deep Q	wealth	+77.81	89.5	0.354	7.4	0.0140	0.505
Deep Q	risk	+52.90	43.1	0.178	5.4	0.0082	0.416
PPO	wealth	+159.97	94.0	0.759	7.3	0.0250	0.671
PPO	risk	+21.48	128.9	0.108	0.7	0.0037	0.107

Table 5.6: Wealth-change versus risk-aware reward ($\lambda = 0.10$) on the held-out months, ten-feature state, six seeds.

The DQN result gives the clearest evidence of the effect of the risk-aware reward. For Deep Q-learning, the mean drawdown falls from 0.0140 under the standard reward to 0.0082 with the risk-aware reward, a reduction of approximately 41%. However, its return also falls from +$77.81 to +$52.90, a reduction of about 32%. Exposure falls from 0.354 to 0.178, meaning that the agent decides to hold only half as much of the asset.

The results do not suggest improved risk-adjusted performance. Instead, the reward mainly encouraged the agent to reduce its exposure to the asset, and the Sharpe ratio also decreased from 0.505 to 0.416.

The PPO results show the same effect more strongly. With the risk-aware reward, PPO's exposure fell to 0.108, and its average trading activity dropped below one trade per month, from 7.3 to 0.7. As a result, its mean wealth change decreased from $159.97 to +$21.48.

REINFORCE shows no noticeable change. Its results are identical for both rewards. This is consistent with the earlier behavioural findings: REINFORCE policies do not change their trading behaviour simply by changing the reward.

5.7.3 Interpretation of the risk-aware return

The risk-aware reward changed the behaviour of the state-dependent agents, but there is no clear evidence that it led to better risk management. Instead, the results suggest that the penalty mainly encouraged the agents to reduce their exposure to the asset.

These results do not show that risk-aware reinforcement learning is ineffective in general. However, they do show a limitation of the reward formulation used in this study. An agent's policy could reduce drawdown to zero by simply holding less of that asset or by never entering the market, but this would not satisfy the main objective of generating useful investment returns.

A more effective objective would therefore need to consider both return and risk, rather than mainly penalising exposure to drawdown. The relevant question becomes whether the agent can reduce drawdown while still leading to meaningful returns and exposure. For example, the objective could reward the agent for achieving higher returns for a given level of risk. Chapter 6 discusses other possible alternatives in future work.

5.8 Transaction-Fee Sizing

Transaction costs are an important part of the trading environment because frequent trading can reduce the returns produced by a strategy, even when individual decisions appear reasonable. A strategy may appear profitable before fees are included but become less attractive once the cost of each trade is taken into account.

The main experiments use a transaction fee of 0.05%. To test whether the observed behaviour changes as this fee varies, the fee was swept from 0 basis points (0.00%) to 50basis points (0.50%) on the validation months with the nine-feature state, and everything else held fixed.

The sweep was run on all algorithms, each with six freshly trained seeds per fee level. Table 5.7 and Figure 5.7 report the outcome.

Fee	B&H ΔW	REINF. ΔW	REINF. trades	Deep Q ΔW	Deep Q trades	PPO ΔW	PPO trades
0bps(0.00%)	+$177.09	+$177.56	5.0	+$108.00	10.9	+$155.35	4.5
5bps(0.05%)	+$166.92	+$154.96	10.3	+$86.16	8.5	+$118.55	3.7
10bps(0.10%)	+$156.76	+$143.74	4.7	+$77.60	8.9	+$78.50	2.7
25bps(0.25%)	+$126.33	+$88.76	10.3	+$32.35	3.9	+$26.40	1.2
50bps(0.50%)	+$75.83	+$41.31	2.8	+$16.32	1.9	+$12.60	1.0

Table 5.7: Transaction Fee sizing on the validation months; six seeds. Trades are per month. Fees are shown in basis points and as a percentage of trade value.

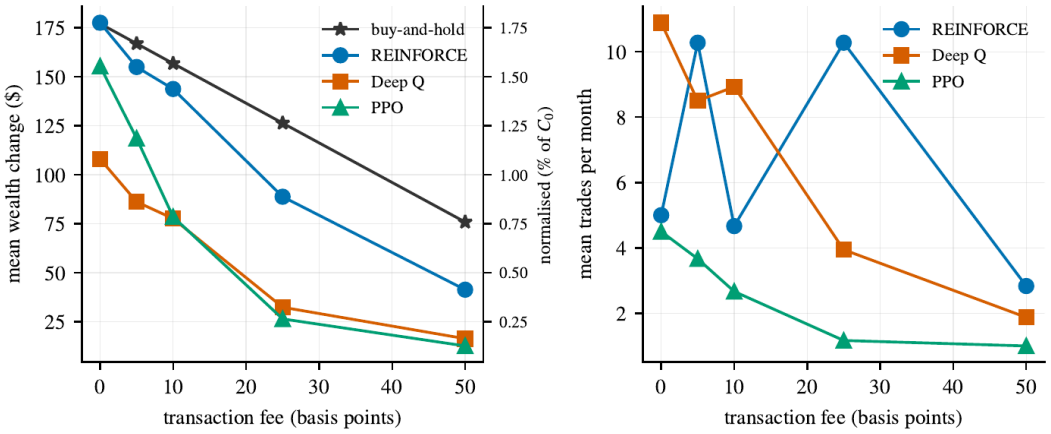

Figure 5.7: Transaction Fee sizing from Table 5.7. Left: average wealth change falls for every strategy as fees rise. Right: average trades per month as fees rise.

5.8.1 Effect of transaction costs on returns & trading behaviour

As expected, higher transaction fees reduce the returns of the different algorithm strategies. The effect is important for strategies that trade frequently because more of their portfolio gains go to transaction costs.

Deep Q-learning shows the clearest reduction in trading activity as the fee of taking a trade increases. Its average number of trades falls from 10.9 per month at 0bps (0.00%) to 1.9 at 50bps (0.50%), an 83% reduction. As each action gets more expensive, the agent acts less, reducing exposure. This is consistent with the algorithm correctly incorporating transaction costs into its environment.

REINFORCE does not show the same clear pattern. Its trade counts vary, running from 5.0, 10.3, 4.7, 10.3, and 2.8, respectively, with no consistent decrease in trades taken as the fee rises. This aligns with the earlier state-independent results: an agent that does not rely on information from its current state to make decisions is less likely to adjust its behaviour when trading costs change.

PPO's average trade count also falls from 4.5 to 1.0, showing that many seeds are pushed to stop trading due to higher fees. This is also why the average return collapses by 77%, from +$177.56 to +$41.31, a difference of $136.25 per month, even though the average trade count moves only gently.

5.8.2 Interpretation

The transaction-fee experiment therefore reflects another difference between the learning algorithms in terms of their change in behaviour. Deep Q-learning changes its trading behaviour as trading costs increase, even though its overall returns are below the buy-and-hold benchmark. REINFORCE produces returns generally closer to the buy-and-hold strategy, and it also shows higher exposure that does not respond consistently to changes in transaction costs. PPO, on the other hand, shows a major collapse in returns as fees increase, while its trade counts decline more gently. Overall, the learned agents lose proportionally more because they trade more often in this experiment.

This indicates that the learning system is capable of responding to different changes when the learned policy is sufficiently state-dependent.

5.9 Discussion of the Main Findings

The experiments produced several findings about both the performance of the learning agents and the way they behaved within the trading environment. They also answer the research questions of Section 5.1

5.9.1 The Benchmark Remains Difficult to Beat

The main finding is that none of the learned agents could outperform the simple buy-and-hold benchmark on the held-out test period. Buy-and-hold achieved a wealth change of $180.25, compared with $171.51 for REINFORCE as the best learned agent, while every other agent fell further behind with $127.35 for PPO, and $74.68 for DQN.

However, it is important to note that the test period consists of generally favourable market conditions. Therefore, the experiments cannot simply be viewed as evidence that reinforcement learning failed, as staying invested in generally rising markets is already a strong strategy

Buy-and-hold is therefore a difficult benchmark to improve upon. A learned strategy that reduces exposure must therefore focus on periods when leaving the market is advantageous enough to compensate for the return not gained while being out of the market.

The experiments provide limited evidence that the tested agents could make this decision and overall did not demonstrate such behaviour.

5.9.2 Additional State Features Did Not Lead to Better Performance

The state comparison did not show a consistent advantage from adding drawdown to the nine-feature state. This is important because the state representation was designed to include information about market movements, portfolio position, wealth, and risk.

For REINFORCE and DQN, the changes in return were -$4.12 and $3.13, respectively, in the nine-to-ten-feature comparison. The lack of improvement in performance does not necessarily mean these features were poorly selected. Instead, their usefulness depends on whether the learning algorithm can use them to make better decisions.

This does not imply that the nine-feature state is universally sufficient. It means that, in this particular environment and with this wealth-change objective, additional market information produced no measurable improvement.

The later risk-aware experiments explain the agents' change in behaviour when the objective explicitly included drawdown, but this mainly led to reduced market participation.

5.9.3 The Algorithms Differed in Their Use of the State

One of the clearest differences between the algorithms is behavioural rather than financial.

REINFORCE and PPO often produced policies that were classified as state-independent. Hence, their relatively strong returns need to be carefully interpreted. A policy that buys whenever possible can perform well during a period of rising prices without making significant use of the market information provided to it.

Deep Q-learning showed a different pattern. It consistently produced state-dependent behaviour in 6 of 6 seeds, indicating that its decisions changed with the state currently being observed. However, this did not lead to higher returns. Instead, the agent generally reduced its exposure to the asset.

This result shows why financial return alone is not enough to evaluate the learned trading policies from reinforcement learning algorithms. Without the behavioural analysis, the strong performance of REINFORCE could be mistaken for evidence that the model had learned a useful relationship between market information and trading decisions.

5.9.4 Lower Risk Was Mainly Associated with Lower Exposure

The results show a consistent relationship between market participation and risk; specifically, risk was reduced mainly by participating less in the market.

Deep Q-learning achieved lower drawdown than buy-and-hold, but this was due to lower exposure, and overall, it led to lower returns in the same proportion.

This means lower drawdown should not be interpreted as evidence that DQN avoided the worst market movements, as this differs from identifying and avoiding specific periods of downside markets.

A proper risk-management strategy would ideally remain invested during rising market periods while reducing exposure during market downsides once detected. The current results do not provide enough evidence to conclude that the agent learned this type of selective risk management.

5.9.5 The Trade Size Affects Performance

The trade size experiment shows that the transaction size is part of the problem definition rather than merely an implementation design.

A small trading size limits how much exposure an agent can build during a monthly episode. This creates a limitation when comparing the learned agent with the simple buy-and-hold strategy. If the environment prevents the agent from investing enough in the asset, poor performance may stem from trade size rather than the quality of the learned policy.

Under the initial $0.10C_0$ setting, the attainable exposure was considerably lower than that of the buy-and-hold benchmark, reaching only 0.690 average exposure as compared to 0.910 for buy-and-hold

The $0.25C_0$ configuration addresses this limitation by increasing the agent's attainable exposure to 0.838, giving a fairer basis for comparing the learned policies with the benchmark, while retaining more control over the positions held.

The trade size experiments also suggest that changing the trade size can have a larger effect on performance than adding features to the state. This supports the decision to calibrate trade size accurately before working on the main learned strategies.

5.9.6 Transaction Costs Provided a Useful Behavioural Test

The transaction-fee experiment showed that Deep Q-learning reduced its trading activity as transaction costs increased, suggesting that the agent responds to the financial costs of its actions within the environment.

Deep Q-learning's trading activity reduced from 10.9 trades per month with no transaction fee to 1.9 trades at a 0.50% transaction fee, representing an approximately 83% reduction. REINFORCE, however, did not show the same response.

This provides further evidence that Deep Q-learning was making greater use of the information available through the environment. However, this additional responsiveness did not result in better financial performance.

5.9.7 Variation Between Seeds Is Important

This experiment showed considerable variation between random seeds, especially for the policy-gradient methods.

In some configurations, different seeds produced noticeably different behaviours. Some policies turned towards high exposure, while others traded less or became more conservative.

A seed producing a strong return would not be enough to conclude that the algorithm had learned a reliable trading strategy. Consistent behaviour across multiple seeds proves that, and becomes stronger evidence that, the observed result is derived from that algorithm rather than a product of one training run.

5.9.8 Implications for the Research Objectives

Overall, the experiments provide a qualified answer to the main research question of whether reinforcement learning algorithms can learn profitable trading strategies for a portfolio (in this case, the SPY asset).

The reinforcement learning agents learned trading policies, but the experiments did not show they could outperform the simpler buy-and-hold benchmark on unseen data.

The results also show that learning behaviour strongly depends on the algorithm, with clear differences between the learning methods. REINFORCE moved towards remaining invested and made little use of state information. DQN, however, consistently changed its actions based on the observed state, making it highly state-dependent, but it did not yield higher returns.

PPO behaviour was closer to REINFORCE than to DQN. Although PPO uses an actor-critic structure and a clipped policy objective to stabilise updates, its performance was less consistent across seeds, with a larger spread in returns. This suggests that PPO's additional stability mechanisms did not produce an effective trading policy in this environment.

The risk-aware reward also changed the behaviour of the state-dependent agents and reduced drawdown, but the reduction was mainly due to lower market exposure rather than better timing for entering and exiting markets during downward periods. When the penalty became too large, the agents almost stopped trading altogether.

These findings suggest that the main challenge identified by the experiments is not just related to providing the agent with more information. Instead, the results strongly point to the interaction between the reward function, trade size, and the behaviour learned by the agent as key factors in determining whether the resulting strategy is useful.

5.10 Chapter Summary

The chapter first determines the final trade size because this affects how quickly an agent can build or reduce its position. It then examines whether the learned strategies rely on the information from the state representations, followed by a comparison between the main nine-feature state and the additional risk-aware state. It then presents the main held-out test results, followed by experiments on the risk-aware reward and the effect of transaction costs. The chapter concludes by bringing the findings together and relating them to the research objectives.

The results show that none of the learned agents consistently outperforms the buy-and-hold benchmark over the held-out dataset. The behavioral analysis offers more insight into this outcome. The policy-gradient algorithms rely less on available state information, whereas the value-based agent uses more available state information but generally has a lower exposure. The experiments also indicate that the fixed trading amount affects performance. Finally, the drawdown-based reward reduces both return and exposure, suggesting that the risk-aware reward mainly encourages lower market participation rather than generating better risk-adjusted trading performance.

References

[1] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[3] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[4] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[5] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[7] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Machine Learning*, vol. 8, no. 3–4, pp. 293–321, 1992.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[9] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proceedings of the 35th International FLAIRS Conference*, 2022.

[10] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[12] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[13] M. L. de Prado, *Advances in Financial Machine Learning*. Wiley, 2018.

[14] W. F. Sharpe, "The Sharpe ratio," *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994.

[15] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018, pp. 3207–3214.